Software Architecture and Design

Oscar Chaparro

Too simple?

THE SUBMISSION



Challenge: a CLI tool that calls an API and pages through the data. One submission: ~30 lines of Python, the simplest way to meet the requirements. ✓ all requirements

THE SECOND REVIEWER

Proposed as the reference for a 10/10. The other reviewer wouldn't pass it to an interview — not enough sophisticated language features. **"Too simple."**

REJECTED

Why simple wins: every part is a way to fail

CLI → API

API down → 1 way to fail

CLI → cache → retry layer → API

API down | stale cache | state out of sync | retry storm

→ many

"Great software design looks simple because it eliminates as many failure modes as possible during the design stage."

S. Goedecke, "Great software design looks underwhelming," Mar 2025 · seangoedecke.com/great-software-design

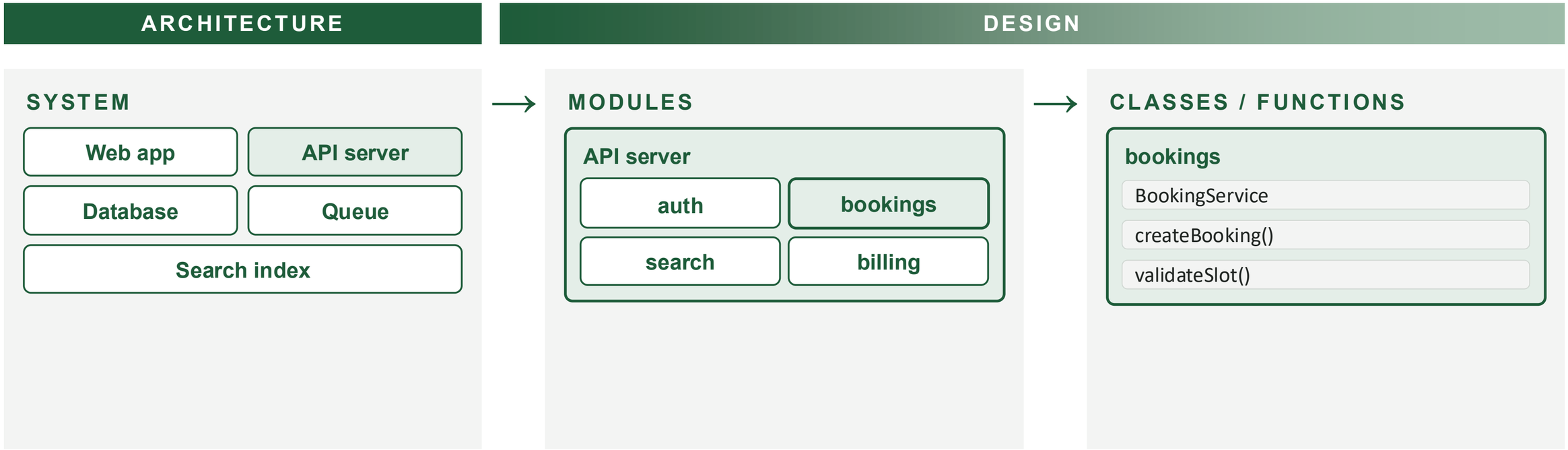
Agenda

1. What architecture and design are
2. Design principles
3. From quality attributes to design decisions
4. Architectural styles
5. Designing inside an existing codebase
6. Recording design decisions
7. Agents and design

What architecture and design are

Structure, decisions, and reasons

Architecture and design



Architecture: the main parts of a system, how they connect, and the key decisions behind them — **and why**.

Design: the same questions at a smaller scale.

One system, several views

Module view



How is the code split, and what may depend on what? → **where do I make a change?**

Chromium layers · VS Code

Runtime view



What runs, and how do the parts talk while it runs? → **what fails together?**

Chromium processes · VS Code extension host

Deployment view



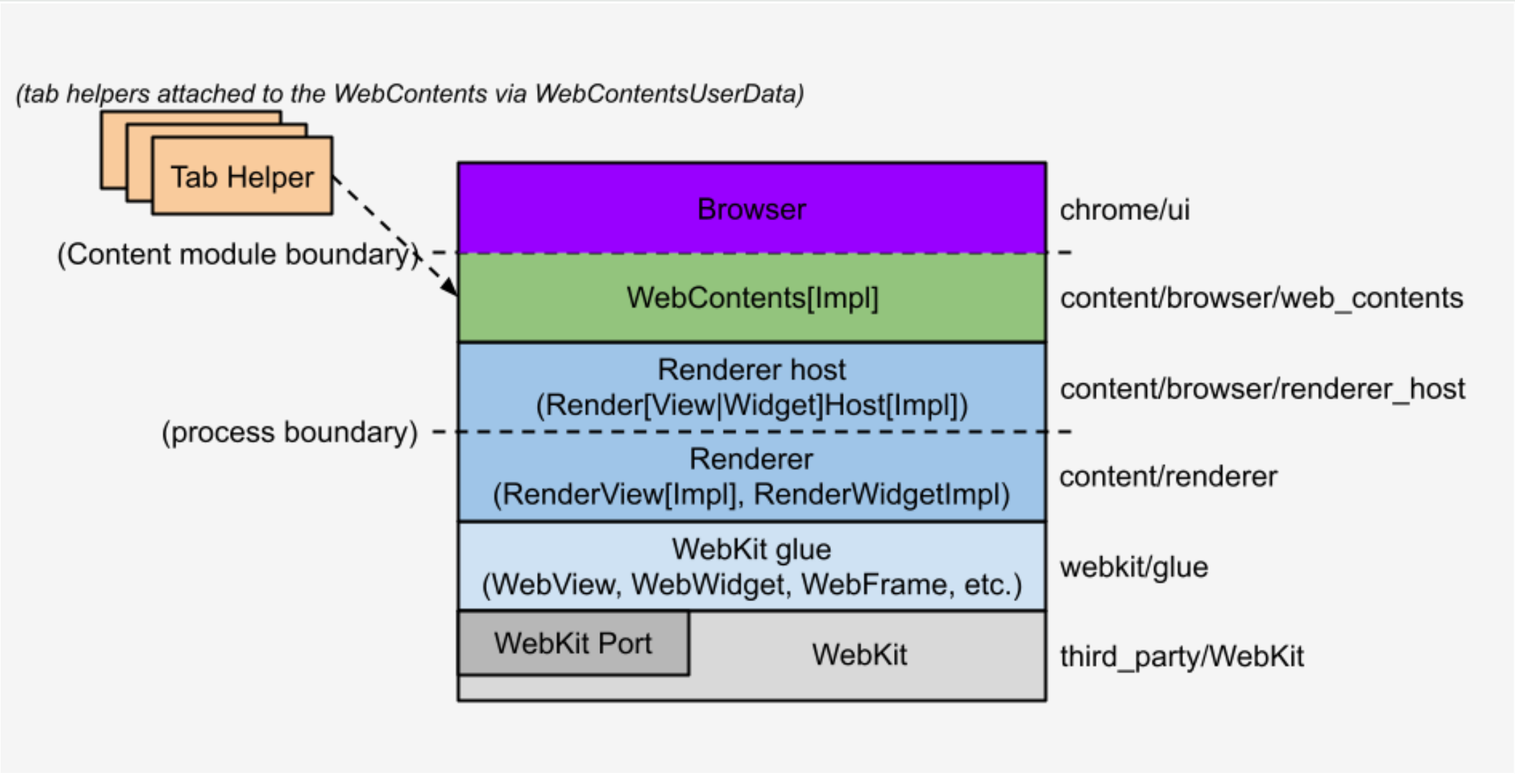
Which machines does it run on? → **where does it run, what does it cost?**

Stack Overflow

Different questions → different views. All of them are architecture.

Chromium: module view

MODULE VIEW



Source: chromium.org — “How Chromium Displays Web Pages” · CC BY 2.5

KEY COMPONENTS

WebKit (+ Port)
rendering engine; the Port connects it to OS services (loading, graphics)

Glue
converts WebKit types to Chromium types — the “WebKit embedding layer”

Renderer / Render host
carry commands and notifications across the process boundary

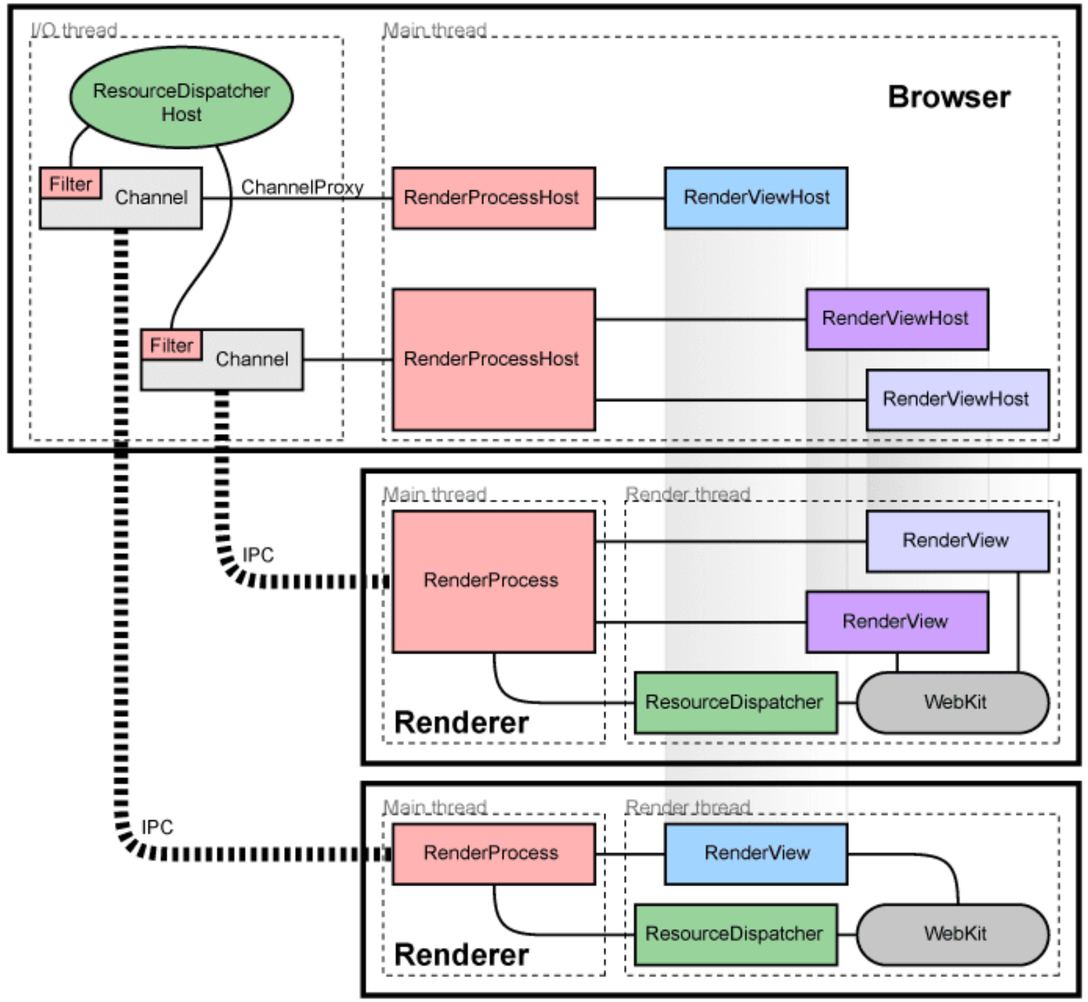
WebContents
reusable component that renders HTML into a view

Browser
the browser window; holds many WebContents

Tab helpers
small add-ons attached to a WebContents (favicons, infobars)

Rule: no layer depends on a higher layer.

Chromium: runtime view



Source: chromium.org — “Multi-process Architecture” design document · CC BY 2.5

RUNTIME VIEW

KEY COMPONENTS

Browser process
one; runs the UI

Renderer process
one per site; runs WebKit in a sandbox

RenderProcessHost
browser-side handle on a renderer

Channel + IPC
async messages between processes

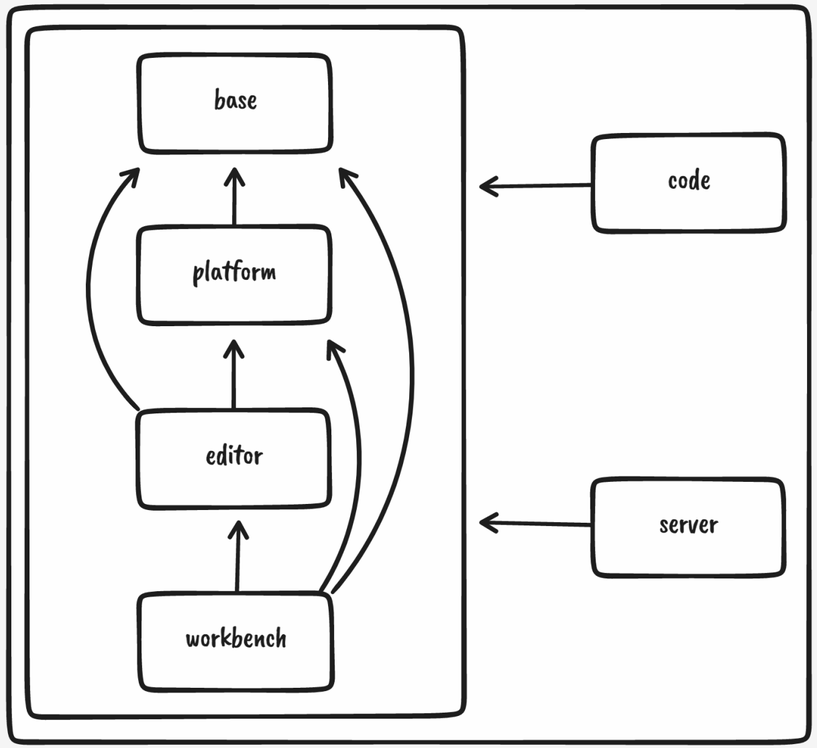
ResourceDispatcher (Host)
network requests go through the browser

Why: a crash shows a “sad tab”, not a dead browser. **Cost:** more memory.

IPC = inter-process communication: messages between processes, which don’t share memory.

VS Code: module view

MODULE VIEW



Source: github.com/microsoft/vscode/wiki — “Source Code Organization”

KEY COMPONENTS

base
general utilities and UI building blocks

platform
service injection and shared services

editor
the Monaco editor core

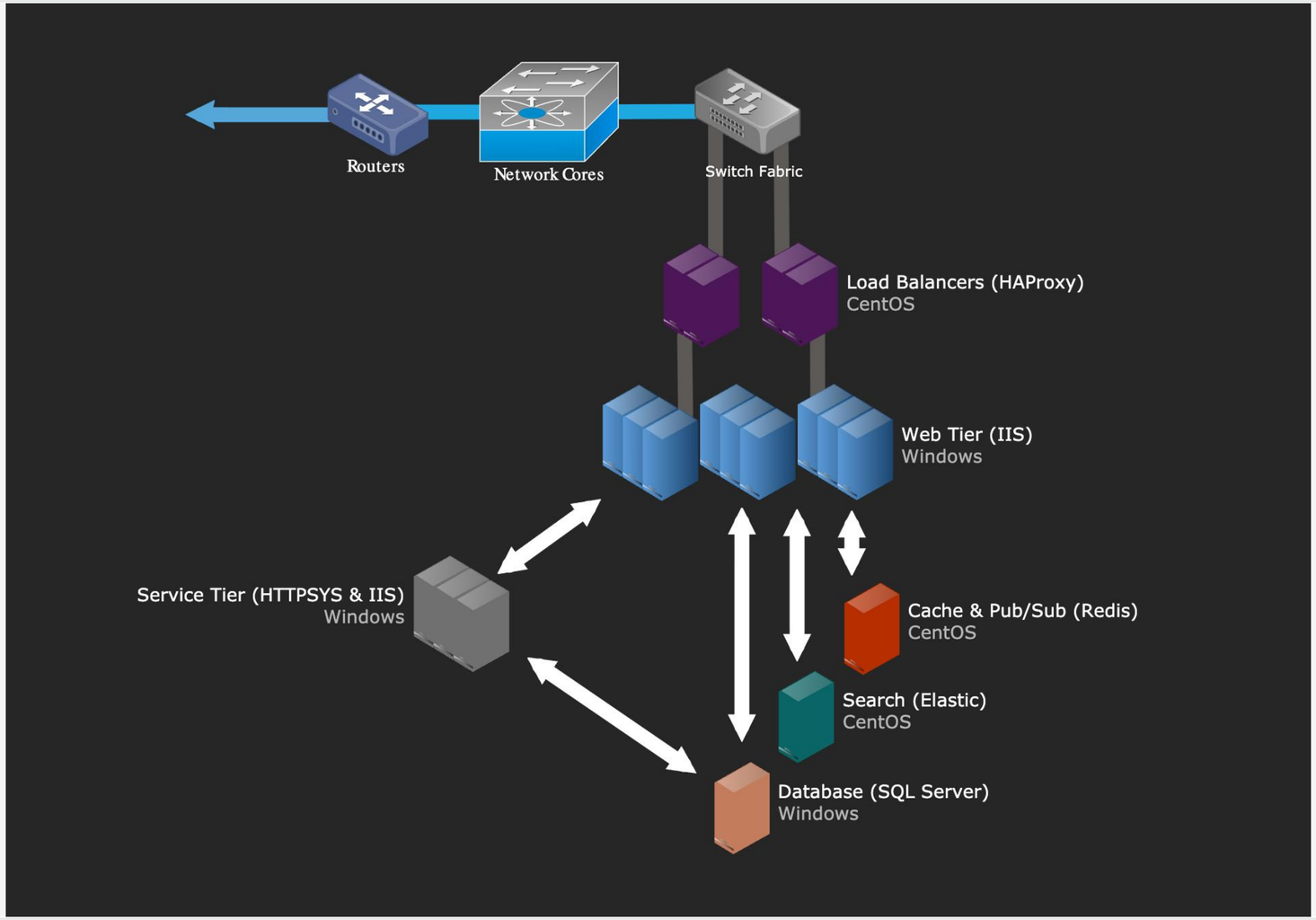
workbench
the IDE UI: editors, panels, status bar

code / server
starts the app: Electron main process, windows, CLI / remote server

extension host
separate process; extensions use only the extension API

Rule: a layer uses only the layers below it. Extensions never touch the core directly.

Stack Overflow: deployment view



Source: N. Craver, “Stack Overflow: The Architecture – 2016 Edition,” nickcraver.com

DEPLOYMENT VIEW

KEY COMPONENTS

Load balancers (HAProxy)
send each request to a web server

Web tier (IIS, ASP.NET MVC)
the main application that serves the sites

Service tier
separate servers for the tag engine and internal APIs — heavy work kept off the web tier

SQL Server
the main data store

Redis
shared cache and pub/sub between servers

Elasticsearch
search

A huge site, mostly one application, on few servers — 2016 snapshot.

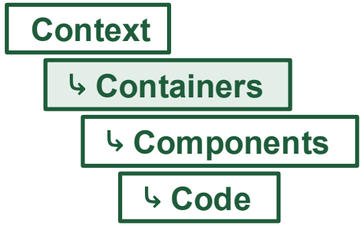
How people draw architectures

Boxes and arrows



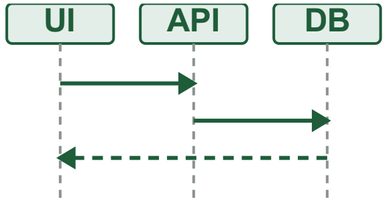
most common; fast, but what does an arrow mean?

C4



four zoom levels: context → containers → components → code

UML



mostly sequence, component and deployment diagrams

Diagrams as code

```mermaid
graph LR
  web --> api
  api --> db
```

Mermaid, PlantUML, Structurizr — kept in the repo

Whatever you use, add a legend: say what a box and an arrow mean.

C4 in brief

KEY FACTS

Simon Brown, c. 2011

4 zoom levels: context → containers → components → code

Fixed abstractions: person, system, container, component

Free notation: any shapes, plus a key and labels

Extra views: landscape, dynamic, deployment

POPULAR TOOLS

Diagrams as code

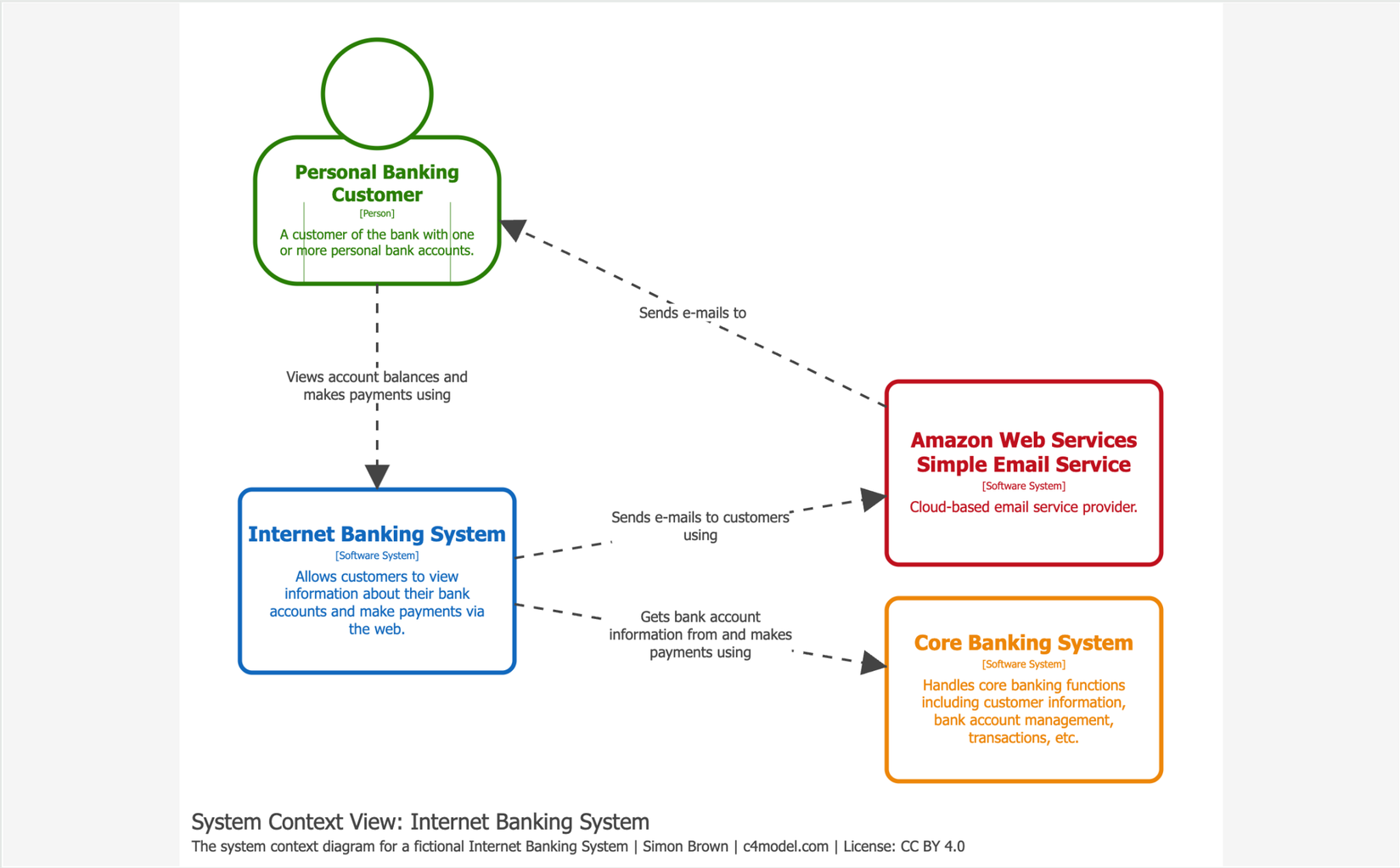
C4-PlantUML | Structurizr | Mermaid

Drawing tools

draw.io | IcePanel | Lucidchart / Miro

Source: c4model.com

C4: system context diagram



Source: Simon Brown, c4model.com · CC BY 4.0

LEVEL 1 · CONTEXT

KEY COMPONENTS

Person
a user or role (the customer)

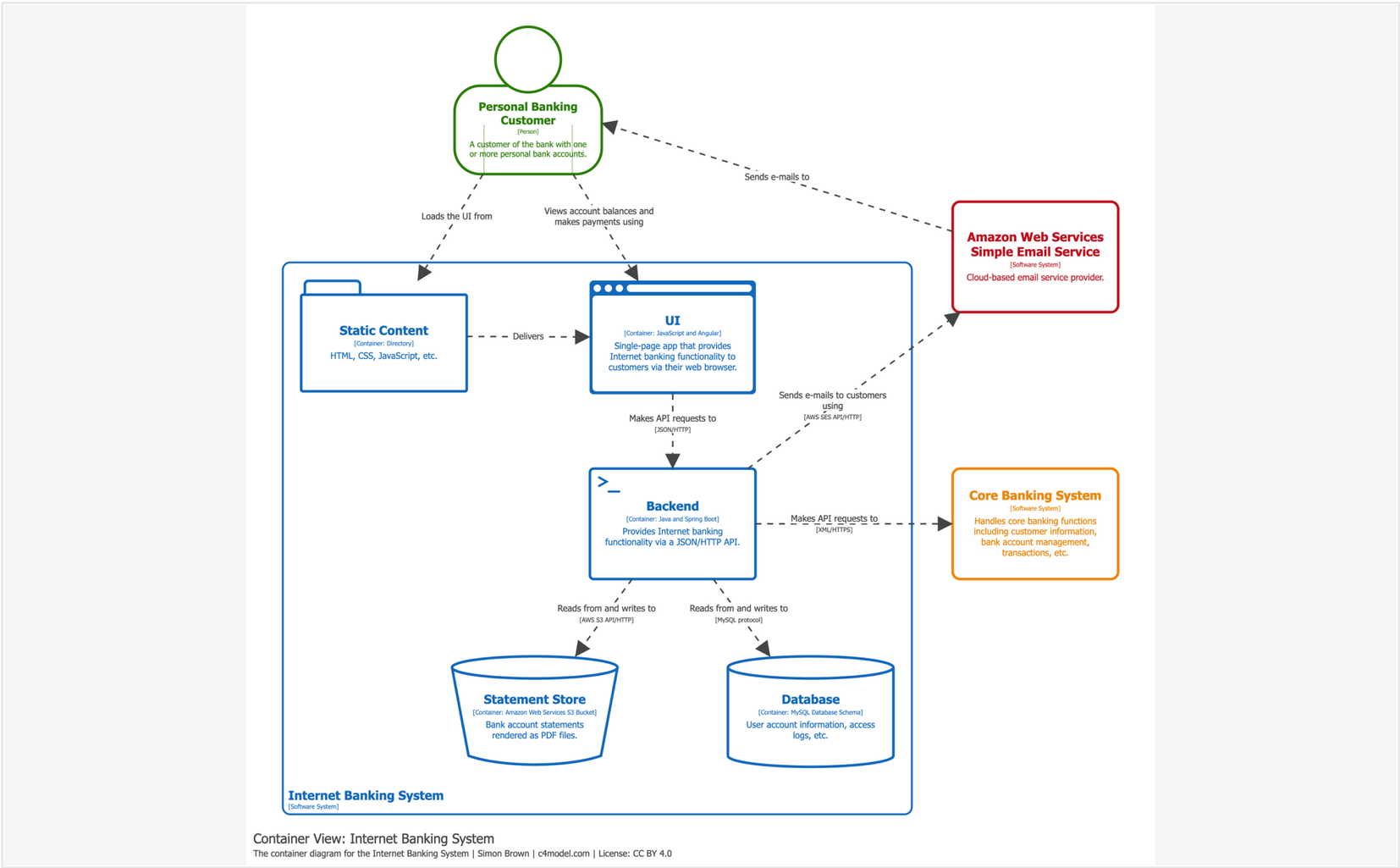
Software system (in scope)
the system we are building: Internet Banking

Existing systems
outside systems it talks to: mainframe, e-mail

Arrows
labelled with what happens

Big picture: no technology yet.
Readable by non-technical people.

C4: container diagram



Source: Simon Brown, c4model.com · CC BY 4.0

LEVEL 2 · CONTAINERS

KEY COMPONENTS

Container

an application or data store that runs on its own

Web app, single-page app, mobile app

what customers use

API application

server-side logic

Database

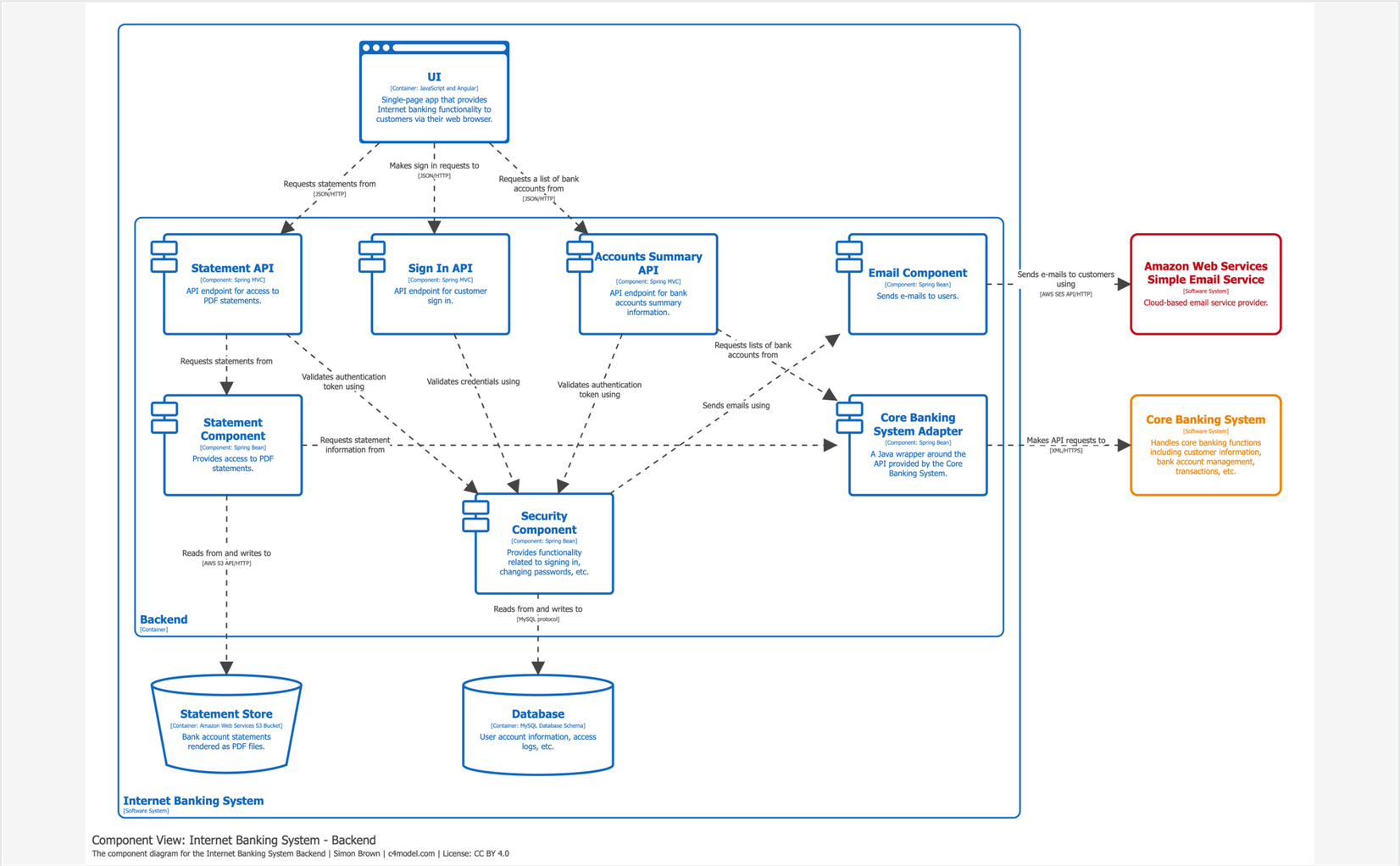
stores the data

Labels

each box names its technology; each arrow its protocol

Zoom in: the main parts, their technology, and how they talk.

C4: component diagram



Source: Simon Brown, c4model.com · CC BY 4.0

LEVEL 3 · COMPONENTS

KEY COMPONENTS

Container in scope
the API application, drawn as a boundary

Components
its main parts: controllers, security, facades

Responsibilities
each box says what it does and its technology

Arrows
calls between components and to containers

Zoom in again: inside one container. Optional.

UML in brief

KEY FACTS

Booch, Rumbaugh, Jacobson; OMG standard since 1997

Fixed notation: every shape has a defined meaning

14 diagram types: structure and behavior

In industry: a few diagrams, used informally

POPULAR TOOLS

Diagrams as code

PlantUML | Mermaid

Drawing tools

draw.io | Lucidchart | Visio

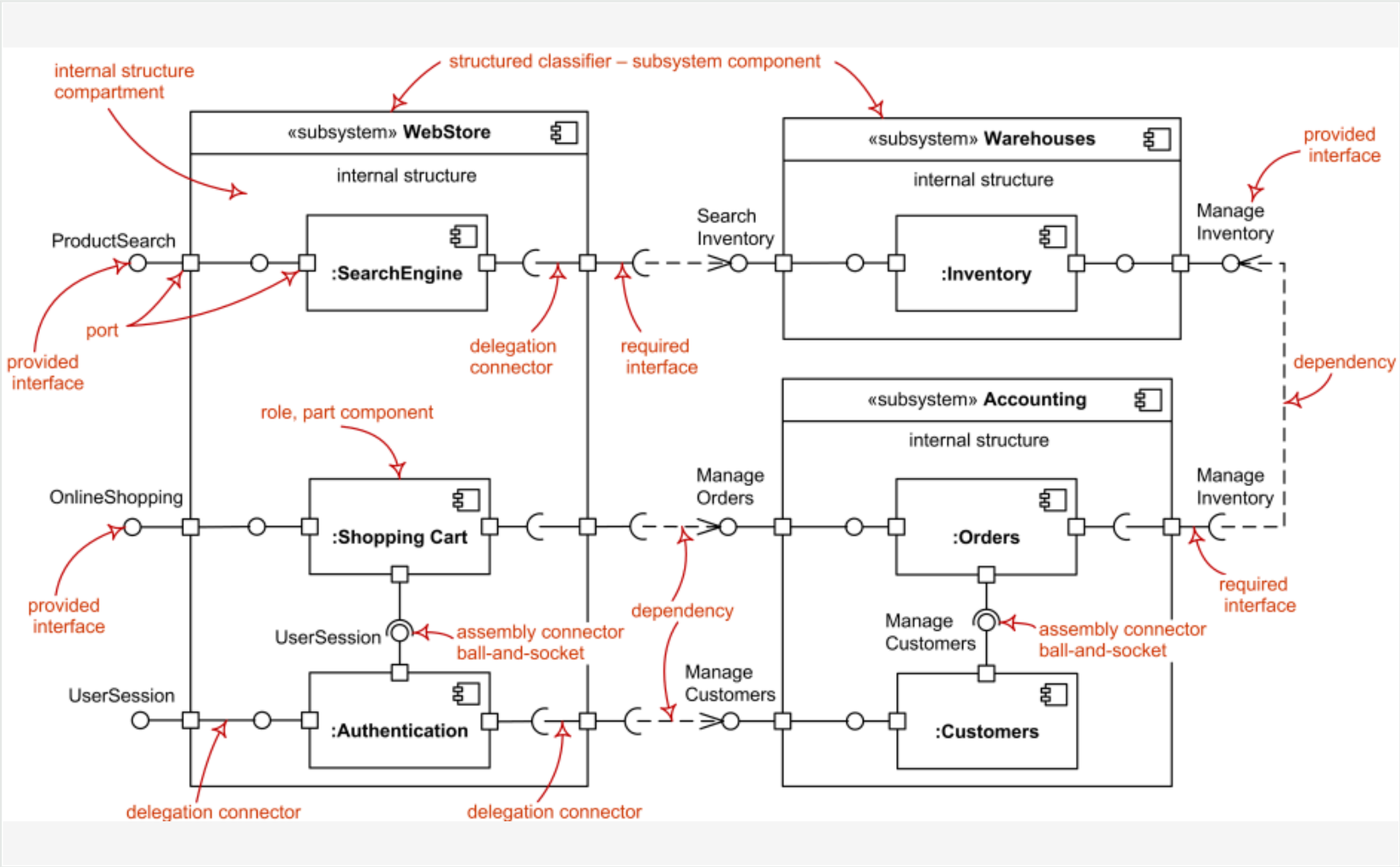
Modeling tools

Enterprise Architect | Visual Paradigm | StarUML

UML standardizes the notation. C4 standardizes the abstractions.

Source: omg.org/spec/UML

UML: component diagram



Source: K. Fakhroutdinov, uml-diagrams.org — “UML Component Diagrams Reference”

UML 2

KEY COMPONENTS

Subsystem / part

«subsystem» WebStore contains parts like :SearchEngine

Provided interface (ball)

a service offered, e.g. ProductSearch

Required interface (socket)

a service needed, e.g. SearchInventory

Port

small square where interfaces attach

Delegation / assembly connector

port → inner part / ball meets socket

Dependency (dashed arrow)

one subsystem uses another’s interface

WebStore uses Warehouses and Accounting only through their interfaces.

Quality attributes drive the structure

Feature: “Find documents by a word.”

↓ TWO ARCHITECTURES ↓

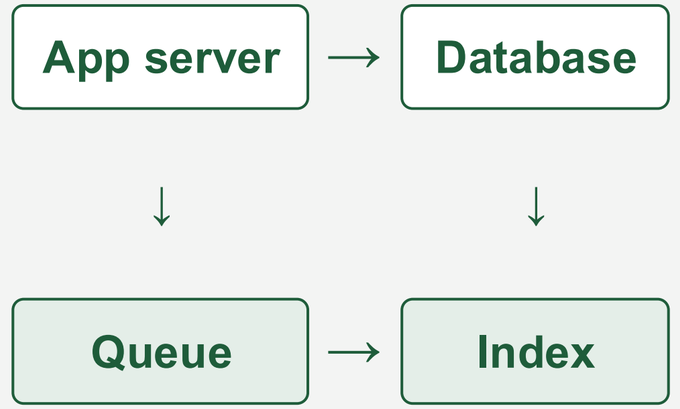
A few hundred documents



→ one query on the database

scale: small

Millions of documents, results in < 500 ms



→ full-text index, background indexing

performance: < 500 ms at millions of docs

Two laws

1 TRADE-OFFS

Everything in software architecture is a trade-off.

Cache: faster reads ↔ one more component to keep in sync

Separate processes: a crash stays contained ↔ more memory

Search index: fast search ↔ results lag behind updates

2 WHY?

Why is more important than how.

The code shows how, not why.

You can see the queue, but not that it keeps search under 500 ms.

Without the why, nobody can safely change it.

Someone removes the “unnecessary” queue and search slows down.

The why is what you re-check when things change.

If the reason no longer holds, the design can change.

— Richards & Ford, *Fundamentals of Software Architecture*

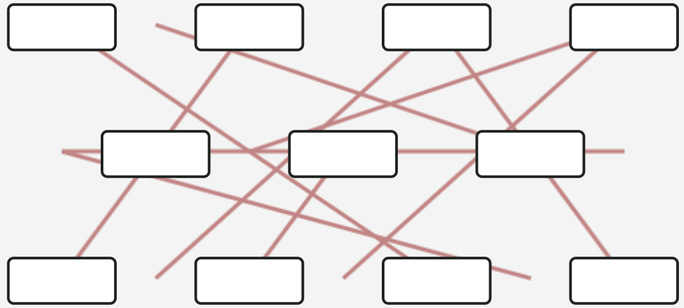


Good design goes unnoticed



✓ 0 incidents

Good design: nothing happens. Changes are easy. Looks obvious afterwards.



✗ pager: incident #14

Bad design: impressive, many parts, clever tricks — and frequent incidents.

“Good design is self-effacing: bad design is often more impressive than good.”

— S. Goedecke, 2025

Design principles

Ideas that make change cheaper



Modular design

WITH GOOD MODULES

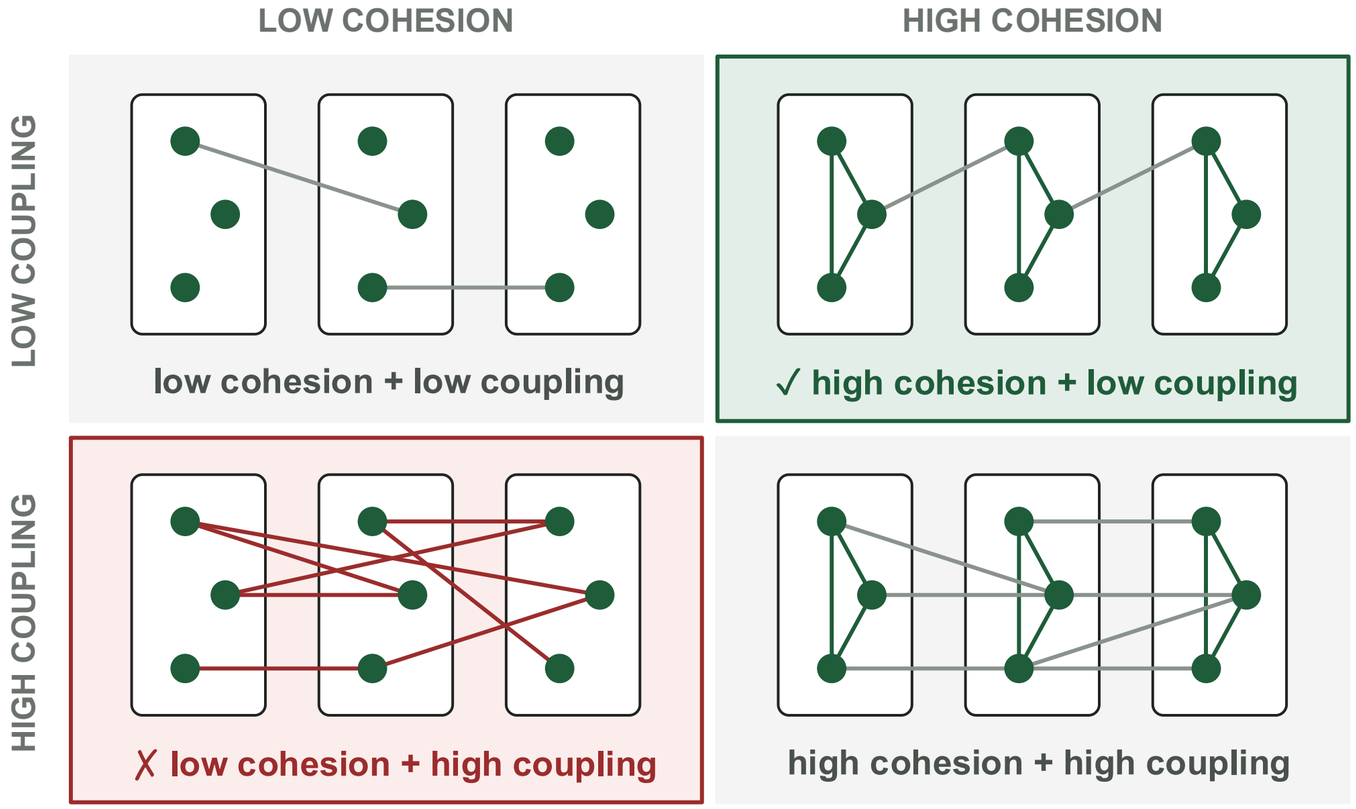
- ✓ Understand one part at a time
- ✓ Test parts on their own
- ✓ Reuse parts elsewhere
- ✓ Replace a part easily
- ✓ Fix and extend at lower cost

WITH TIGHT COUPLING

- ✗ A change ripples into other modules
- ✗ Can’t test a module without its dependencies
- ✗ Can’t reuse a module on its own
- ✗ Putting modules together takes more effort



Coupling and cohesion



Coupling: how much modules depend on each other → **keep low**

Cohesion: how much a module's parts belong together → **keep high**

Defined at function and class level; the same ideas apply to packages, services and systems.

Gitea team: a small feature meant changing many files. Why?

ENABLES changing, testing and reusing one module on its own

PREVENTS ripple effects and hidden dependencies between modules

Types of coupling

Worst (top): high coupling **Best (bottom): low coupling**

1	**content**	One module reaches into another’s internal data or code
2	**common**	Modules share global data
3	**control**	One module passes a flag that steers another’s logic
4	**stamp**	A whole record is passed when only a few fields are used
5	**data**	Only the needed data is passed, as parameters

Content coupling (worst)

COUPLING worst **content** → common → control → stamp → data best

Content coupling: one module reaches into another's internals.

BEFORE

```
# customers.py  (module A)
class CustomerList:
    def __init__(self):
        self._items = []

# lookup.py  (module B)
def find_or_add(customers, name):
    for c in customers._items:   # reads internals
        if c.name == name:
            return c
    c = Customer(name)
    customers._items.append(c)   # edits A's data
    return c
```

FIX

```
# customers.py  (module A)
class CustomerList:
    def __init__(self):
        self._items = []
    def find(self, name): ...
    def add_customer(self, name): ...

# lookup.py  (module B)
def find_or_add(customers, name):
    c = customers.find(name)
    if c is None:
        c = customers.add_customer(name)
    return c
```

CONSEQUENCE CustomerList can't change how it stores customers without breaking lookup.py.



Control coupling

COUPLING worst **content** → **common** → **control** → **stamp** → **data** best

Control coupling: a flag tells another module which path to take.

BEFORE

```
def handle(order, action):
    if action == "ship":
        ...  # pick, pack, label, notify
    elif action == "refund":
        ...  # payment, stock, email
    elif action == "cancel":
        ...  # release stock, close order

# caller
handle(order, action="refund")
```

FIX

```
def ship(order): ...
def refund(order): ...
def cancel(order): ...

# caller
refund(order)
```

CONSEQUENCE Three unrelated jobs behind one flag: each new action changes handle(), and callers must know its modes.

Reducing coupling doesn’t mean zero connections. It means depending on less, and on more stable things.



Data coupling (best)

COUPLING worst **content** → **common** → **control** → **stamp** → **data** best

Data coupling: only the needed data is passed. (Before: stamp coupling.)

BEFORE (STAMP)

```
def late_fee(loan):
    days = (date.today() - loan.due_date).days
    return max(days, 0) * 0.25

# test: build a whole Loan
# (book, member, dates...)
```

FIX (DATA)

```
def late_fee(due_date, today):
    days = (today - due_date).days
    return max(days, 0) * 0.25

late_fee(loan.due_date, date.today())

# test: two plain dates
late_fee(date(2026,10,1), date(2026,10,5))
```

CONSEQUENCE late_fee needs two dates, not a whole Loan: testable with plain values, reusable for anything with a due date.

Passing the whole object is fine when the function uses most of it.



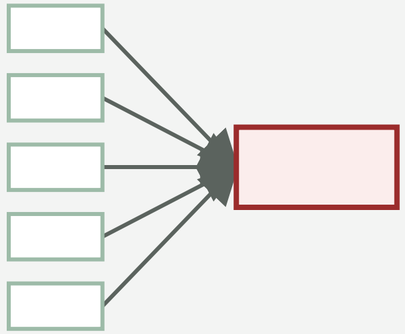
How much coupling?

KIND × AMOUNT



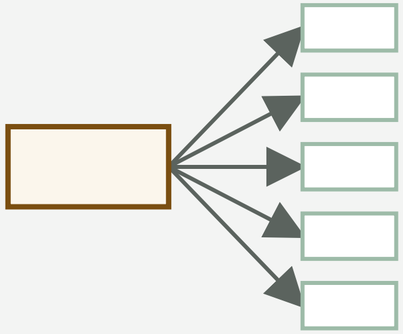
Many weak links still bind modules.

FAN-IN AND FAN-OUT



Fan-in

many depend on it → **risky to change**



Fan-out

depends on many → **fragile**

Coupling = kind × amount.

Types of cohesion

Worst (top): low cohesion **Best (bottom): high cohesion**

1	**coincidental**	Parts are unrelated; grouped by chance
2	**logical**	Parts are logically related but do different things
3	**temporal**	Parts run at the same time (e.g., startup)
4	**procedural**	Parts are related only by execution order
5	**communicational**	Parts work on the same data
6	**informational**	Separate operations on one data structure
7	**functional**	Every part contributes to one task

Coincidental cohesion (worst)

COHESION worst **coincidental** → logical → temporal → procedural → communicational → informational → functional best

Coincidental cohesion: the parts have nothing in common.

BEFORE

```
# joes_stuff.py
def win2lin(path):
    ...

def days_since_epoch():
    ...

def output_report(data, stream):
    ...
```

FIX

```
# paths.py
def win2lin(path): ...

# dates.py
def days_since_epoch(): ...

# reports.py
def output_report(data, stream): ...
```

CONSEQUENCE Hard to find, hard to reuse: importing one function drags in the rest.



Logical cohesion

COHESION worst coincidental → **logical** → temporal → procedural → communicational → informational → functional best

Logical cohesion: parts are logically related but do different things (here, chosen by a flag).

BEFORE

```
# output.py
def output(kind, data, stream):
    if kind == "finance":
        ...  # format report
    elif kind == "weather":
        ...  # format forecast
    elif kind == "number":
        ...  # format number
```

FIX

```
# finance_report.py
def write(data, stream): ...

# weather.py
def write(forecast, stream): ...

# numbers.py
def write(n, stream): ...
```

CONSEQUENCE Unrelated changes pile up in one function, and the kind flag is control coupling.



Functional cohesion (best)

COHESION worst coincidental → logical → temporal → procedural → communicational → informational → **functional** best

Functional cohesion: every part of the module contributes to one task.

EXAMPLE

```
# shipping.py
RATES = {"local": 4.0, "national": 7.5}

def shipping_cost(order):
    weight = total_weight(order.items)
    zone = zone_for(order.address)
    return RATES[zone] * weight

def total_weight(items): ...
def zone_for(address): ...
```

WHY IT'S GOOD

- ✓ One purpose: compute the shipping cost
- ✓ Test it: give an order, check a number
- ✓ Change rates or zones in one place
- ✓ Reuse or replace it as a unit



What is an interface?

An **interface** is what a module exposes to others: its **signatures** (names, inputs, outputs) and its **contract** (what it requires and what it promises).

Signature : names, parameter and return types, errors. A type checker can verify it.

Contract : preconditions (caller ensures), postconditions (module guarantees).

FUNCTION

```
shipping_cost(order)
  -> float
```

order has items;
returns ≥ 0

CLASS

```
CustomerList
  .add()  .find()
```

find() returns None
if absent

MODULE

```
payments
  charge()  refund()
```

refund ≤ amount
charged

SERVICE

```
POST /bookings
```

same key →
no second booking

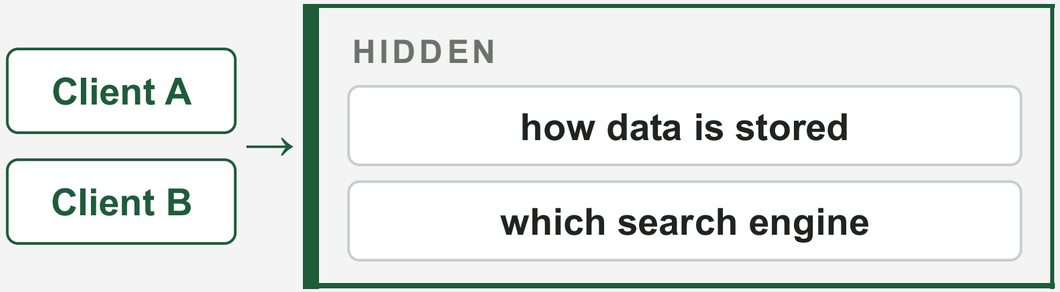
Stable interface: its signature and contract rarely change, even when the implementation does.

Information hiding

CONCEPT

Hide each **design decision likely to change** inside one module, behind a stable interface.

Encapsulation is the mechanism: keep data and code together and expose only the interface (in Python, by convention: _items).



— D. Parnas, 1972

EXAMPLE

```
# documents.py
class DocumentStore:
    def save(self, doc): ...
    def search(self, word): ...
    # hidden: SQLite or Postgres,
    #         query or search index

# caller
store.search("invoice")
```

Swap the database or add an index: callers don’t change.

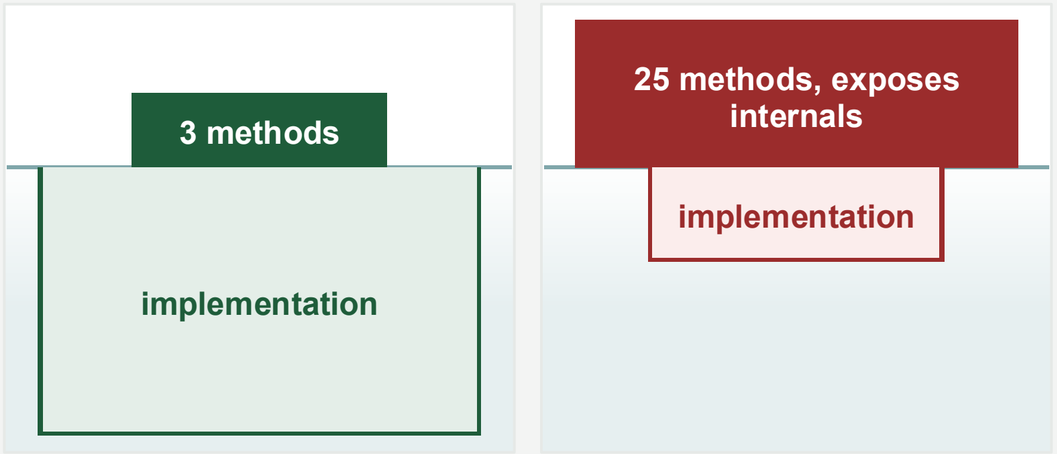
ENABLES changing a hidden decision without touching clients

PREVENTS content coupling: clients depending on internals

Deep interfaces

CONCEPT

A good interface is **small** and hides a lot.



✓ Deep ✗ Shallow

— J. Ousterhout, *A Philosophy of Software Design*

EXAMPLE

SHALLOW: CALLERS RUN THE STEPS

```
r = FileReader(path)
r.open_descriptor()
r.set_buffer_size(4096)
data = r.decode_bytes(r.read_chunk())
r.close_descriptor()
```

DEEP: ONE CALL

```
data = open(path).read()
```

Fewer things to depend on, fewer things to break.

Related: **orthogonality** (unrelated things change independently) · **interface segregation** (small, focused interfaces)

ENABLES using a module without learning its internals

PREVENTS stamp coupling: clients tied to more than they use

Depend on interfaces

Dependency inversion

The core depends on an **interface** , not on Stripe. Stripe depends on that interface.

Open/closed

Open to new providers, **closed** to edits: adding one doesn't change the core.



Medusa: payment → Stripe · file → local S3 · auth → email GitHub Google

Enables: adding a provider without editing the core

Prevents: lock-in to one vendor

Cost: an extra layer. Worth it only if you expect more providers

Medusa: payment providers

WHAT IT IS

Payment provider: an adapter between Medusa and one payment company.

Every provider implements:

initiate **authorize** **capture** **refund** **cancel**

Other providers:

file: local disk or S3

auth: email, GitHub or Google login

MEDUSA CODE (TYPESCRIPT, SIMPLIFIED)

```
// core: the interface
abstract class AbstractPaymentProvider {
  abstract authorizePayment(input)
  abstract capturePayment(input)
  abstract refundPayment(input)
}

// plugs in
class StripeProviderService
    extends AbstractPaymentProvider {
  capturePayment(input) { /* Stripe API */ }
}
```

```
// medusa-config.ts: chosen by config
providers: [{ resolve: ".../payment-stripe" }]
```

Checkout calls `capturePayment()`. The config decides who handles it.



DRY: Don’t Repeat Yourself

DRY = Don’t Repeat Yourself. Every piece of knowledge has **one authoritative place** in the system: knowledge, not code that merely looks alike. — Hunt & Thomas

COHESIVE, NOT DRY

OrderService
placeOrder()
`TAX_RATE = 0.07`

InvoiceService
issueInvoice()
`TAX_RATE = 0.07`

Rate changes: you must remember both.

DRY, NOT COHESIVE

utils
taxRate()
formatDate()
emailTemplate()

One copy, but the parts don’t belong together.

BOTH

TaxRules
rate()
taxFor(order)

used by OrderService and InvoiceService

One place, and it belongs together.

PREVENTS copies that drift apart — the bug fixed in one copy but not the other



Build less

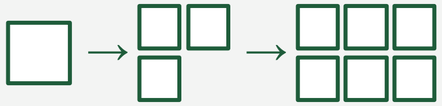
“A complex system that works is invariably found to have evolved from a simple system that worked.” — J. Gall



YAGNI: You Aren’t Gonna Need It

Don’t build it until you need it.

The “future” plug-in system nobody ever uses.



Start simple

Get a simple version working, then grow it.

Rate limiter: count in memory before adding Redis.

smoke detector vs. **no flame**

Remove, don’t just handle

Change the design so the problem can’t happen.

Report takes 20 s → compute it nightly, serve the file.

PREVENTS unused code that still needs maintenance, and complexity nobody needed

SOLID at a glance

S

Single responsibility

one reason to change

SAME IDEA AS

high cohesion

O

Open/closed

add behavior without editing the core

SAME IDEA AS

depend on interfaces

L

Liskov substitution

any implementation can replace another without breaking the code that uses it

SAME IDEA AS

safe to swap

I

Interface segregation

small, focused interfaces

SAME IDEA AS

deep interfaces

D

Dependency inversion

depend on abstractions, not details

SAME IDEA AS

depend on interfaces



Principle or violation?

A **Gitea:** issue search goes through one `Indexer` interface. Bleve, database, Elasticsearch or Meilisearch — chosen in config. The rest of Gitea never knows which indexer is used.

B **Cal.diy:** the function that builds the key that detects duplicate bookings exists in two files.

C **Cal.diy:** Google, Apple, Exchange… each implements the `Calendar` interface (create events, read busy times, etc.). Booking code calls only the interface (easy to add a new calendar vendor.)

D **Gitea:** 65 of 78 files in `routers/web/repo` (web page handlers) import `models` (database code) directly, skipping `services` (business logic).

Which principle does each follow or break?

Information hiding: hide what may change

YAGNI: build only what's needed

Depend on interfaces: not implementations

SOLID: five class-design rules

DRY: one place per fact

Coupling / cohesion: depend on less; group related code

Principle or violation? — answers

	VERDICT	WHY
A	✓ follows	**information hiding / dependency inversion** — search engine is a hidden, swappable decision
B	✗ breaks	**DRY** — if one copy changes, duplicate detection silently breaks
C	✓ follows	**open/closed** — add a calendar without editing the core
D	~ trade-off	**Trade-off between cohesion/coupling/etc.** — relaxed layering: simpler handlers, but business rules can leak into routers

From quality attributes to design decisions

For each NFR: which design moves exist?

A design workflow

1 Quality attribute	2 Remove the problem?	3 Add a mechanism	4 Pay the cost
Say it measurably: “search < 500 ms”	Can a design change or a smaller requirement avoid it? e.g. search within one project, not all 1M documents	If not, add a tactic: cache, queue, retry	Name what it adds: parts, state, failures

WHAT WE VISIT NEXT

Performance & scalability | Reliability | Recoverability | Consistency vs. availability | Security | Interoperability & extensibility

Slow? Try these, in order

Performance & scalability | Remove the problem? | Add a mechanism | Pay the cost

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
1	**Measure** (not a move: find where) find the hot path	"Checkout < 1 s" → **one query takes 80%**
2	**Paginate** never load everything	"History stays fast as it grows" → **50 per page**
3	**Index** let the DB do the work	"Orders by email < 200 ms" → **index on email**
4	**Off the request** background jobs	"Confirm order < 500 ms" → **email + PDF in background**
5	**Scale out** stateless servers, replicas	"10× traffic on Black Friday" → **more servers + replica**
6	**Cache last** adds state: stale data	"Product page < 200 ms, prices 1 min old OK" → **cache 60 s**

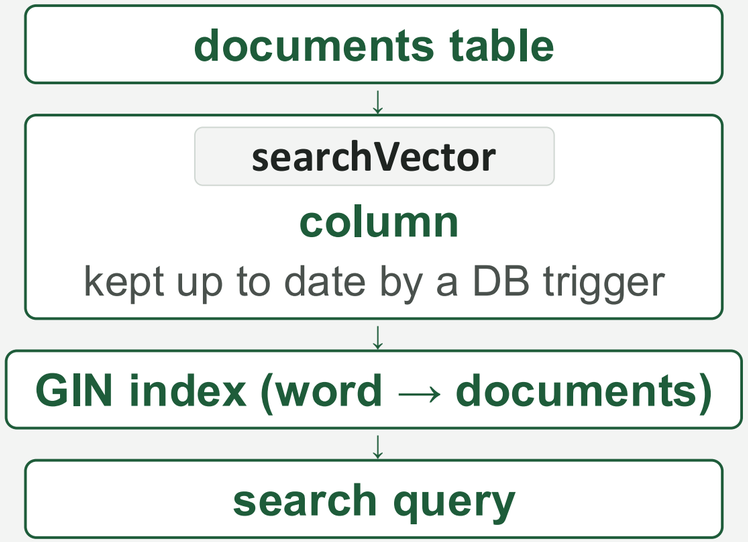


Two projects, two moves

Move 3 · Index

Outline

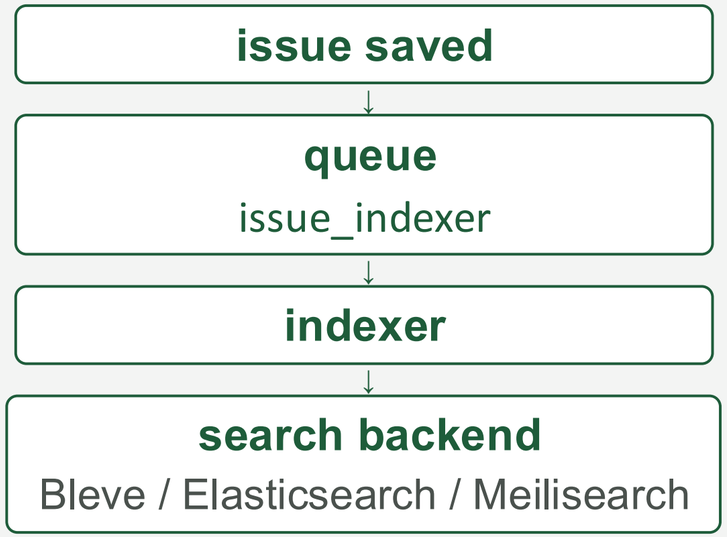
Search looks up words in an index instead of scanning every document.



Move 4 · Off the request

Gitea

Saving an issue returns right away. Indexing happens later, in the background.



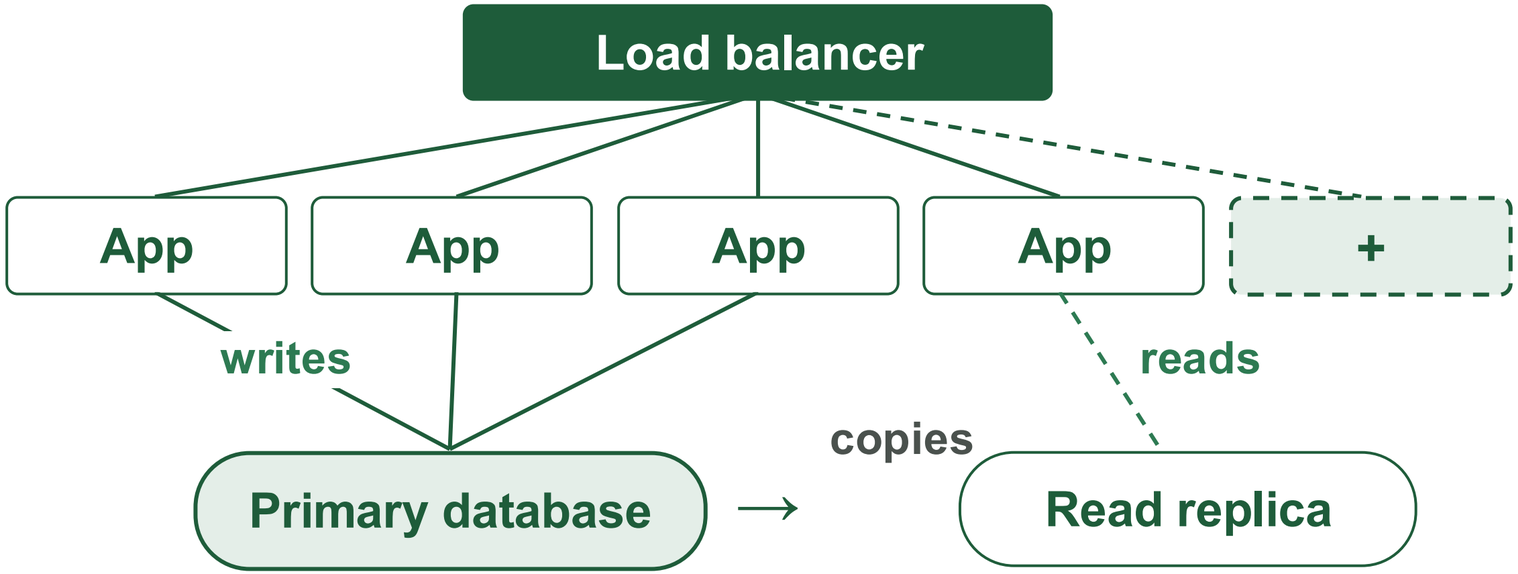
~ Code search is **off by default** : uses a lot of disk

Stateless servers scale out

MOVE 5 · SCALE OUT

Stateless app servers → add more as load grows

State in **one place** → the database (+ replicas)



State

logins, carts, uploads → database, Redis, S3

Why it scales

any server can handle any request

Cost

replicas lag behind; send reads that must be fresh to the primary

Tools: Performance & scalability

Measure	browser DevTools · EXPLAIN ANALYZE · Datadog · Sentry
Load balancing	Nginx · HAProxy · AWS Application Load Balancer
Scale out	Docker · Kubernetes · Google Cloud Run
Background jobs and cache	Celery · Sidekiq · Redis · Cloudflare CDN



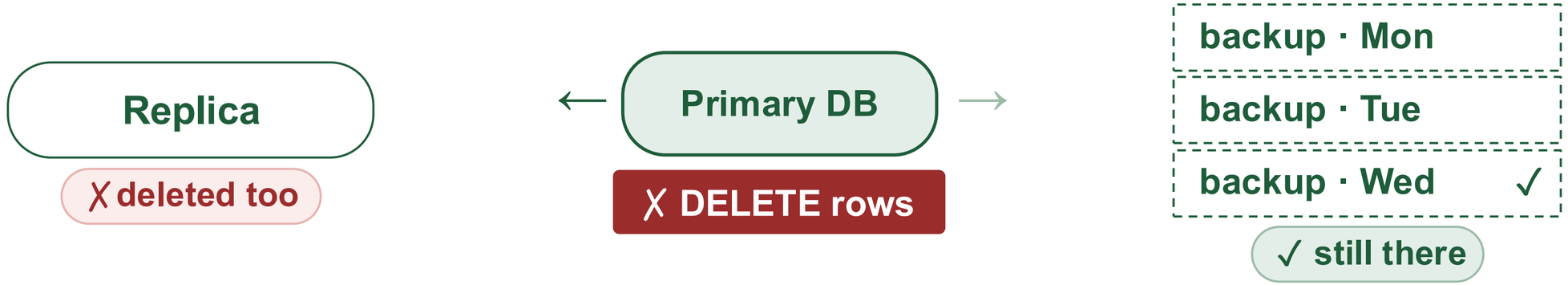
Parts fail? Keep running

Reliability | Remove the problem? | Add a mechanism | Pay the cost

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
SERVER OR DATA FAILS		
1	**Redundancy** replicas take over	“Survive a server failure” → **primary + replica**
2	**Backups** restore after mistakes	“Recover from a bad delete” → **nightly backup, tested restore**
REQUEST FAILS		
3	**Idempotent operations** a repeat has no extra effect	“Double click books once” → **unique key in the database**
4	**Retries with backoff** wait 1 s, 2 s, 4 s…	“Ride out a network blip” → **retry 3 times**
5	**Circuit breaker** adds state, new ways to fail	“Store still works if the reviews service is down” → **after 5 failures, skip reviews for 30 s**

Redundancy: replicas and backups

Reliability	Remove the problem?	Add a mechanism	Pay the cost



Replica: a live copy

Updated: continuously, within ms

Protects against: a machine failure (takes over)

Doesn’t protect: mistakes (it copies them)

Backup: a copy from the past

Updated: on a schedule, e.g. nightly

Protects against: bugs, bad deletes, ransomware

Doesn’t protect: changes since the last backup

Test your backups: restore them regularly.

GitLab.com, January 31, 2017

~300 GB

deleted from the primary database

An engineer deletes the database directory on the **primary**, not the replica

4 ways to recover the data. 3 failed. The 4th was luck.

1 ✗	2 ✗	3 ✗	4 ✓
Replication had stopped	**pg_dump backups** wrong version → empty; alert emails rejected	**Disk snapshots (Azure)** never enabled	**LVM (Logical Volume Manager) snapshot (inside the server)** on staging, 6 h old: luck

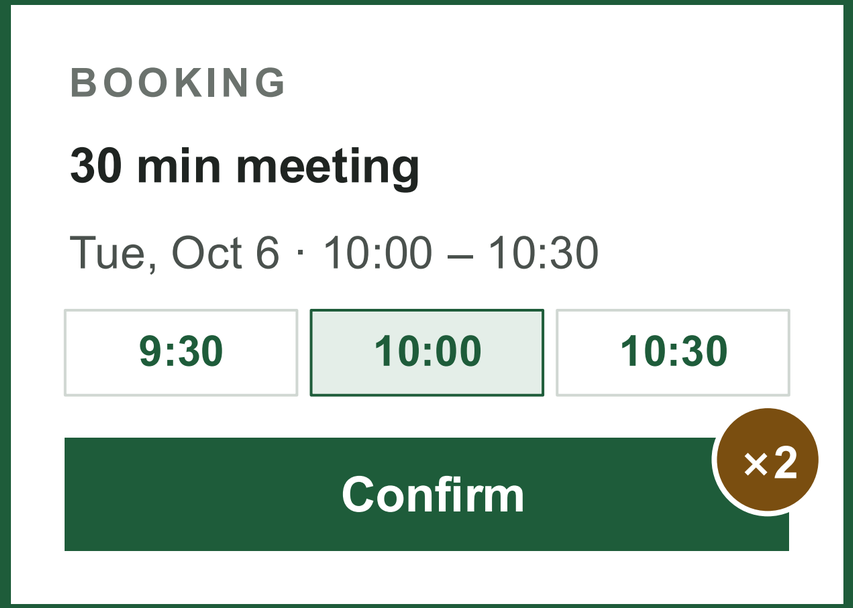
~6 hours of data lost (5k projects/comments, etc.) · ~18 hours to recover



What happens when they click twice?

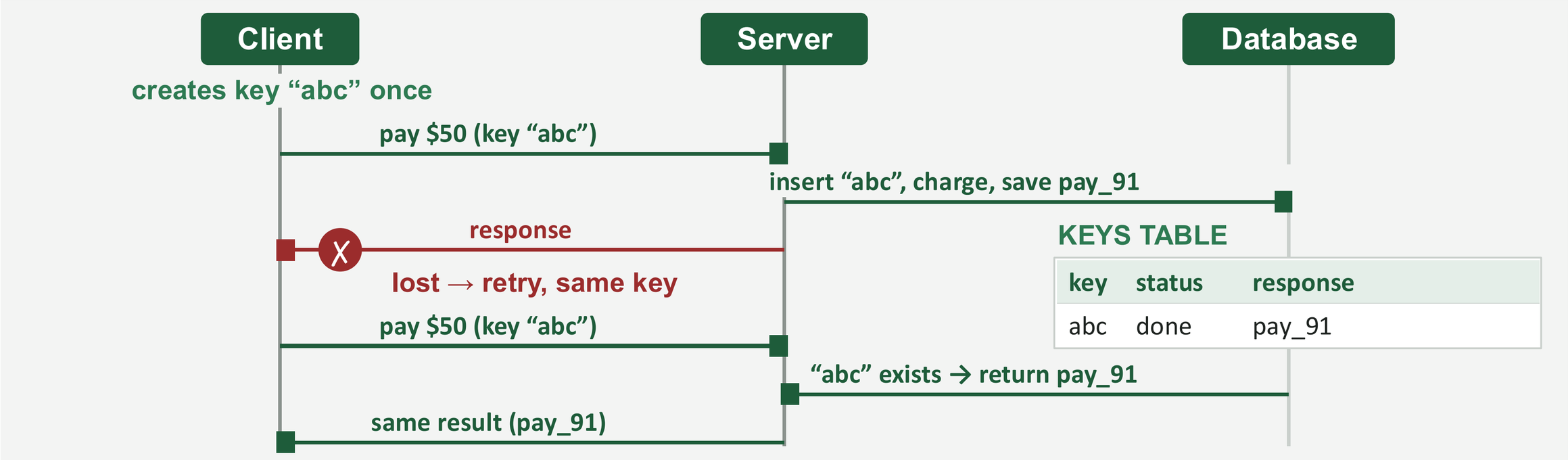
Cal.diy: a booker clicks **Confirm** twice — or the network retries.

How would you make sure the host isn’t double-booked?



Retries need idempotency

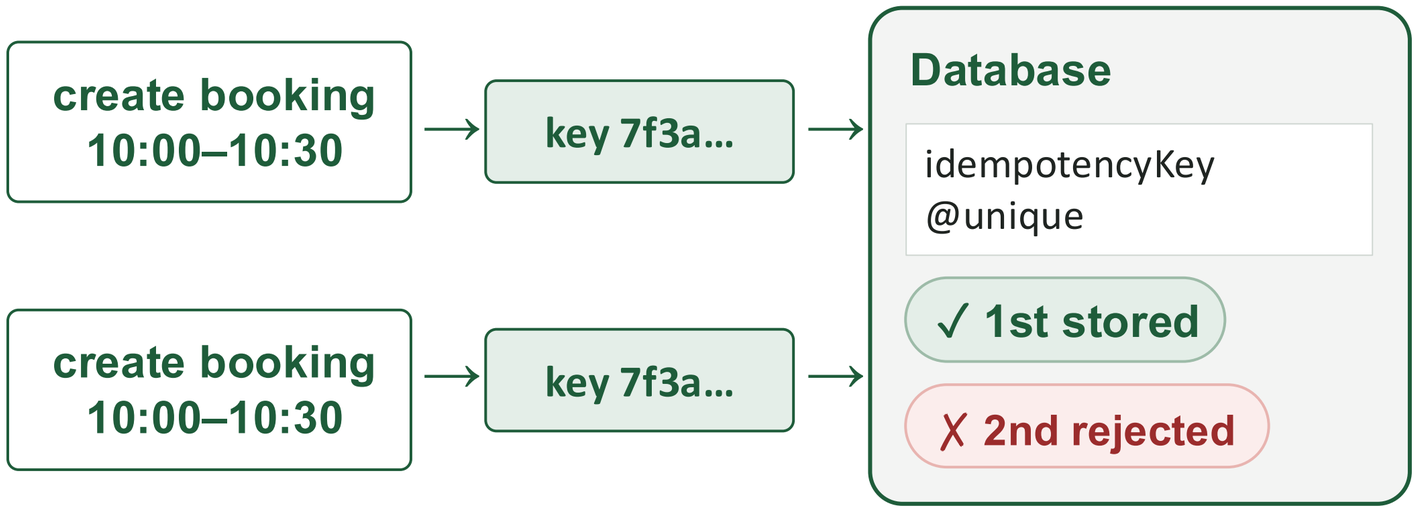
A request times out. Did it happen? **Retry** safely, with an **idempotency key**.



The **database remembers each key and its response** . A unique key column stops two requests from using the same key at once.

Prevents: double charges, duplicate orders and bookings.

What happens when they click twice? — answers



1. **Key = hash(start, end, host)** : computed from the slot, not random, so same slot → same key
2. The database requires a **unique** key: the 2nd insert is rejected
3. Stops double clicks, retries, and two bookers alike
4. Cancel or reject → key cleared → slot free again

Overlapping slots (10:00 vs. 10:15) get different keys; an availability check handles those.

No race, nothing to forget: the problem is removed, not handled.

When a dependency fails

Circuit breaker

The product page calls a **reviews service** . It goes down.

✗ Without: every page waits for a timeout; checkout slows too

Closed — calls go through

Open — skip, use fallback

Half-open — try one call

Closed → Open: 5 failures in a row

Open → Half-open: after 30 s

Half-open → Closed if it works, Open if not

Retries (while closed): wait 1 s, 2 s, 4 s. If they all fail, that's 1 failure for the breaker.

Fail open or closed?

A **check** itself breaks. Let the request through, or block it?

Ask: which is worse, wrongly letting in or wrongly keeping out?

open **let it through**

Rate limiter down: blocking all would take the site down

closed **block it**

Login check down: a short lockout beats exposing data

Decide when designing, not by accident.

Tools: Reliability

Database replicas	Postgres replication · MySQL replication · AWS RDS · Cloud SQL
Database backups	pg_dump · pgBackRest · managed snapshots
Files	S3 versioning · restic
Retries, idempotency, circuit breakers	idempotency keys (Stripe API) · unique constraints · Resilience4j · Polly



Bad change shipped? Limit and undo

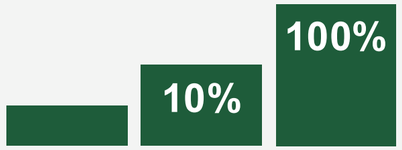
Recoverability	Remove the problem?	Add a mechanism	Pay the cost

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
1	**Prevent** small, tested, automated deploys	“No server left on old code” → **same version everywhere; delete dead code**
2	**Staged rollout** 1% → 10% → 100%	“A bad update hits few users” → **canary first**
3	**Observability** notice early: p95/p99	“Know before users complain” → **alert on p99**
4	**Killswitch** off without deploying	“Stop a bad feature in 1 min” → **feature flag**
5	**Fast rollback** back to the last good version	“Undo a deploy in 5 min” → **one-click rollback**
6	**Postmortem** learn without blame	“Don’t repeat the outage” → **write-up + fixes**



Plan for the bad day

Recoverability | Remove the problem? | Add a mechanism | Pay the cost

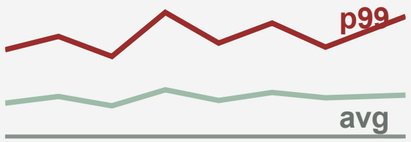


Staged rollout

1% → 10% → 100%

WAYS

- 1% of users first, then more (or by group: employees first, then everyone)
- load balancer sends a little traffic to canary servers
- app store phased release (mobile)



Observability

log why; watch p95/p99, not averages

WAYS

- structured logs with the reason
- metrics + alerts: error rate, p99
- error tracking: stack traces per endpoint



Killswitch

turn a feature off without deploying

WAYS

- config value or environment variable (needs a restart)
- database row or Redis key (instant)
- feature flag service (dashboard, targeting)



p99 = 2 s: 99 of 100 requests finish in under 2 s; the slowest 1 takes longer.

Knight Capital, August 1, 2012

A large US stock-trading firm. Its software sent clients' **orders** (buy or sell shares) to the exchange.



1. **Deploy:** new code on 7 of 8 servers
2. **Market opens:** a reused flag wakes "Power Peg" on the 8th: old code, unused since 2003, never deleted. It can't tell when an order is filled, so it keeps trading
3. **Rollback:** removes the new code but leaves the flag on, so Power Peg runs on all 8

$460M+ **lost in 45 min**

LESSONS

- ✓ Deploy the same version everywhere
- ✓ Delete dead code; never reuse flags
- ✓ Roll back code and config together
- ✓ Have a killswitch

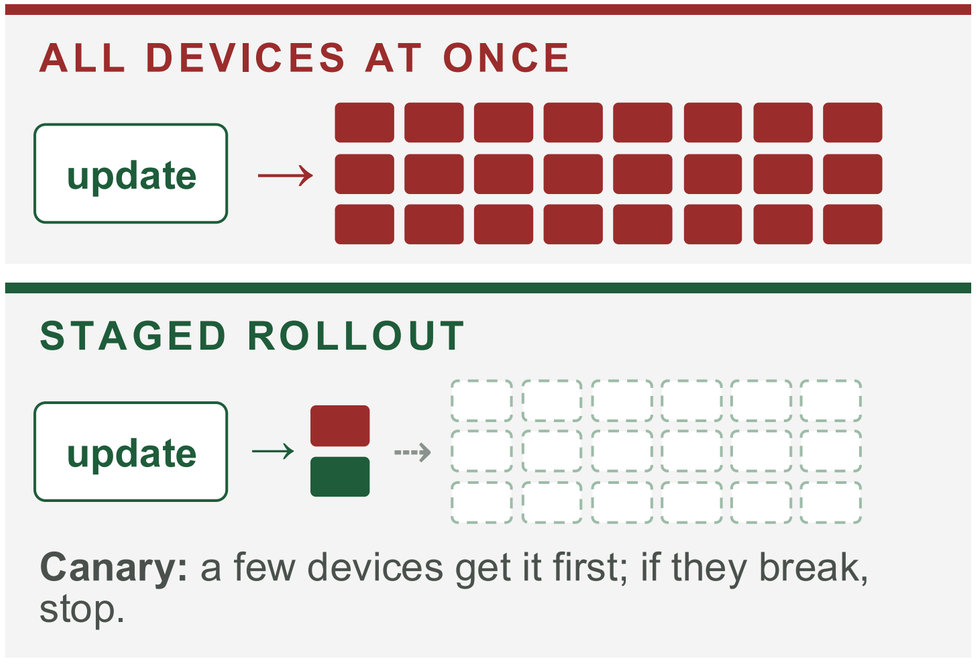
Killswitch: a flag outside the code, checked before the risky action, switched off in seconds.

CrowdStrike, July 19, 2024

CrowdStrike Falcon: security software on millions of company PCs, running inside the Windows kernel. The update: a config file teaching it to spot a new attack.

1. **Update** (treated as data, so sent to all devices at once): the file supplies 21 values; the code expects 20
2. **Crash:** reading the 21st value hits memory the code doesn't own; in the kernel, so Windows crashes on every boot
3. **78 min live:** ~8.5M devices down, fixed by hand

Missed: their tests never filled in the 21st field.



Lessons: roll out in stages · treat config like code: test it with realistic inputs · strictest checks for kernel code · let customers choose when updates arrive

Is recoverability architecture?

Recoverability	Remove the problem?	Add a mechanism	Pay the cost

Design (architecture)

Killswitch hooks in the code

Reversible changes, especially database changes
e.g. add the new column; drop the old one a release later

Code that reports what it's doing

Two versions can run at once (staged rollout)

Practice (process)

Rollout policy: who, when, how fast

On-call and alerts

Rollback drills

Blameless postmortems

Architecture makes recovery *possible*; process makes it *happen*.

Tools: Recoverability

Prevent	CI tests (GitHub Actions) · deploy automation (Argo CD)
Roll out in stages	canary rollouts · Kubernetes rollouts
Notice problems	Prometheus · Grafana · Datadog · Sentry
Turn off	feature flags (LaunchDarkly, Unleash)
Undo fast	rollbacks in CI/CD · Terraform

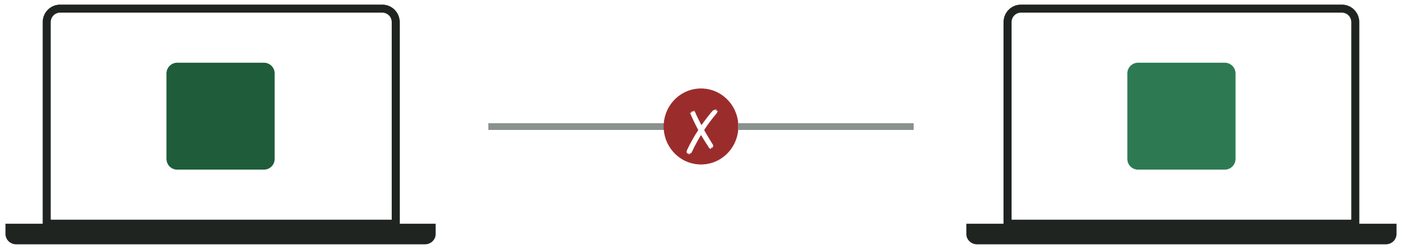


Concurrent edits? Choose how to merge

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
1	**Avoid conflicts** one owner per piece of data	“Only the author edits a post” → **single writer**
2	**Lock or block** one writer at a time	“A transfer never double-spends” → **transaction / lock**
3	**Server decides** last write wins	“Shape properties in a design tool” → **the server applies edits in order; the last one wins**
4	**Version rule** reject the stale edit, or pick the newer version	“Two edits to one record” → **“changed by someone, reload?” or keep the higher version**
5	**CRDTs** conflict-free replicated data types	“Both people’s typing survives” → **merges both edits, even offline**

Two users, one document

Consistency vs. availability | Remove the problem? | Add a mechanism | Pay the cost



When the network splits, choose:

Stay available — accept edits, merge later

A edits B edits → merge

e.g. offline docs, shopping carts, chat

Stay consistent — block edits until reconnected

A edits B read-only

e.g. bank transfers, seat booking, last item in stock

Three ways to merge

Server decides the order



server is the authority; last write wins per property

Figma

A version rule

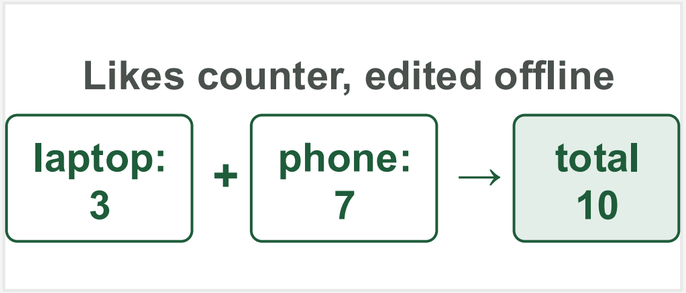


reject the stale edit, or keep the higher version

Rails, Hibernate, HTTP ETags

Excalidraw **Actual Budget**

CRDTs (conflict-free replicated data types)



both edits survive: text, counters

Outline (Y.js)

Both edits must survive? → **CRDT** · **Silent overwrite OK?** → **server decides (no server: newest version wins)** ·

Not OK? → **reject the stale edit**

Excalidraw: who wins?

Each shape has an ID, a **version** (+1 on every edit) and a random **versionNonce**. A remote copy differs from mine: **which one do I keep?**

Editing it? yes →
1 I'm **editing** it → keep mine
only while you edit; versions decide after

↓ no

Versions differ? yes →
2 Higher **version** wins
each edit adds 1: higher = newer

↓ tie

Same version →
3 Same version → lower **versionNonce** wins
a random number; every browser picks the same

EXAMPLE

local v5 · nonce 812

remote v5 · nonce 301

→ **remote wins**

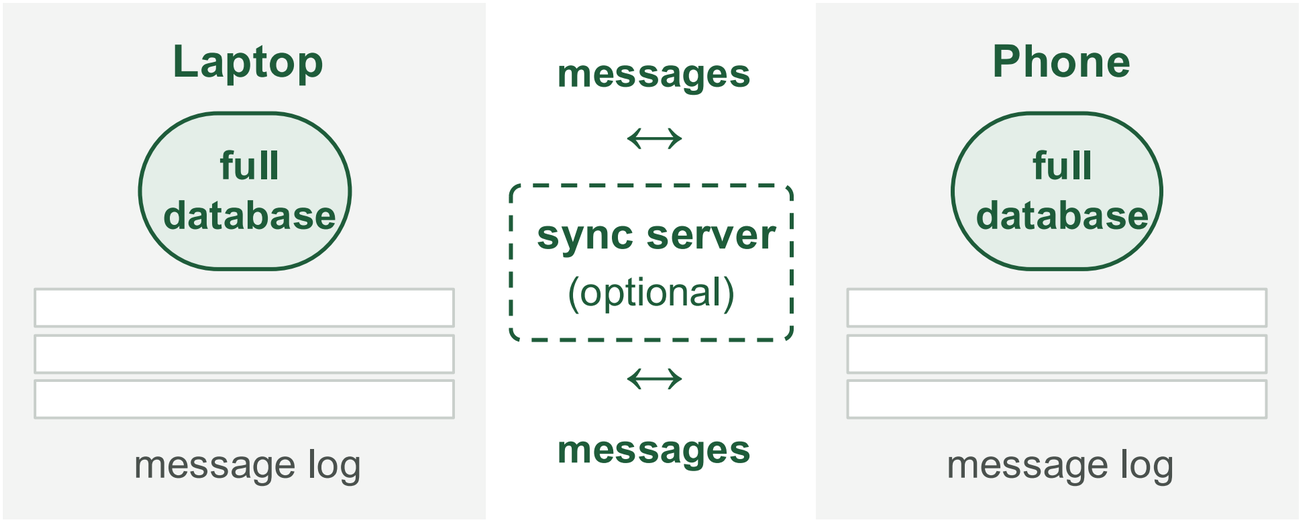
browser ⇄ encrypted **relay server** encrypted ⇄ **browser**

The server can't read the data, so it can't decide: **clients must**

Cost: one of two simultaneous edits is lost. Rare: same shape, same moment, and everyone sees it and can redo it.

Actual Budget: local-first

A personal-finance app. **Local-first:** each device has all the data and works offline; devices sync when they can.



Every device has the **full database**

Each change = a message: **timestamp · table · row · column · value**

(ts, transactions, row 42, amount, -25.00)

1 change = 1 message. **Newest timestamp per cell wins** : a version rule with timestamps, so no server decides. (Technically a simple CRDT.)

works offline **your data stays yours**

How CRDTs keep both edits

Counter

Problem: two devices add likes offline

Idea: one count per device; total = sum

laptop 3 + phone 7 → 10

Set

Problem: Bob removes “milk” (he added it to the cart). Alice, offline, never saw it and adds it.

Idea: same item added and removed at the same time → the add wins

“milk” stays

Text (Y.js)

Problem: positions shift when others type

Idea: each character has a permanent ID; insert “after ID…”; deletes leave hidden anchors

“cat”: Alice adds h after c, Bob adds r after t → “chatr” on both

Merge in any order, any number of times → every copy ends the same. Cost: extra metadata, harder to debug.

How should these merge?

1. **Team task board:** a shared board of cards, one per task (like Trello). Two people rename the same card at the same moment.

2. **Online store admin:** two admins edit the same product’s price.

3. **Shared notes app:** two people type in the same paragraph; one is offline.

4. **Banking app:** two transfers from the same account at the same moment.

Pick one: Avoid conflicts · Lock or block · Server decides · Reject the stale edit · CRDT

How should these merge? — answers

1	**Server decides**	task board: a lost rename is cheap and visible
2	**Reject the stale edit**	store admin: a silent price overwrite costs money
3	**CRDT**	notes app: both people’s text must survive
4	**Lock or block**	banking: money can’t be spent twice

Tools: Consistency vs. availability

Strong consistency	database transactions (Postgres)
Merge concurrent edits	Yjs, Automerge
Offline sync built in	Firebase / Firestore



Under attack? Design for security

Security	Remove the problem?	Add a mechanism	Pay the cost

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
1	**Collect and expose less** what you don't have can't leak	"Customers pay by card" → **let the payment provider store card numbers**
2	**Least privilege** only the access needed	"A report job reads data" → **read-only database user**
3	**LLM output is untrusted** an AI can be tricked; limit what its tools can reach	"AI assistant summarizes my inbox" → **its tool reads only the logged-in user's messages**
4	**Validate input** never trust the client	"Price comes from the form" → **recompute on the server**
5	**Secrets and encryption** out of code; HTTPS	"API key in the repo" → **secrets manager**

Fail closed and least privilege

Security	Remove the problem?	Add a mechanism	Pay the cost



Collect and expose less

what you don’t have can’t leak

- Let the payment provider store card numbers
- Delete logs after 30 days
- Don’t expose admin endpoints publicly



Least privilege

only the access needed

- Report job → read-only database user
- Build pipeline’s key → can deploy one app only
- App’s database user → can’t drop tables

LLM output is untrusted

Security	Remove the problem?	Add a mechanism	Pay the cost



Treat LLM output as untrusted

an AI can be tricked; limit what its tools can reach

✓ `fetch_messages()`

✗ `fetch_messages(user_id)`

- Tools read only the logged-in user's data
- Never run AI-written SQL or shell commands directly
- Ask the user before the AI sends email or pays



Fail closed

a decision for every check: when unsure, deny

- Login service down → deny login
- Fraud check down → hold the payment
- Virus scanner down → reject the upload

Validate input; protect secrets

Security | Remove the problem? | Add a mechanism | Pay the cost



Validate input

never trust the client

- Recompute the price on the server
- Use parameterized queries (blocks SQL injection)
- Limit upload size and type



Secrets and encryption

keep keys out of code

- Keys in a secrets manager, not the repo
- HTTPS everywhere
- Store passwords only as slow hashes (bcrypt)

Tools: Security

Access and login	OAuth / OpenID Connect · Auth0 · Keycloak
Find vulnerabilities	Dependabot · Snyk · CodeQL · Semgrep
Secrets	Vault · AWS Secrets Manager
Encrypt	HTTPS / TLS · Let's Encrypt · bcrypt / Argon2 (passwords)



Others need in? Open the system

	MOVE	EXAMPLE REQUIREMENT → WHAT TO DO
1	**Use a standard** formats others already speak (CSV, iCal, OAuth)	interop "Calendars show my bookings" → **iCal feed**
2	**Public API** others call you	interop "Partners create bookings" → **POST /bookings**
3	**Webhooks** you call others	interop + extend "Notify Slack on a new issue" → **POST to their URL**
4	**Plugins / providers** others add code behind your interface	extend "Add a payment company" → **provider package**
5	**Keep the interface stable** don't change its signature or contract	both "Old clients keep working" → **/v1/, new optional fields**

Ways to open a system

Interoperability & extensibility	Remove the problem?	Add a mechanism	Pay the cost

API

interop

them → you

When: whenever they want

Cal.diy public API

Webhooks

interop + extend

you → them

When: when an event happens

Gitea → Slack

Plugins / providers

extend

you | their code

When: as part of your logic

Medusa → Stripe

A webhook is an HTTP callback: they give you a URL; you call it when something happens.

Once used, an interface is hard to change

POLICY

“We do not break userspace.” (Linus Torvalds, Linux kernel)

→ Linux never breaks programs that run on it, even to fix a bug.

COST

`Referer:` **(misspelled)** in the HTTP spec, 1996

→ Never fixed: millions of servers and browsers depend on the wrong spelling.

WHY

“With a sufficient number of users of an API, … all observable behaviors of your system will be depended on by somebody.” (Hyrum’s law)

→ People rely on what they can see, not just what you documented: the real interface is bigger than the documented one.

Change interfaces by adding, not by changing.



Breaking or not?

Changes to `POST /api/documents.info` :

	CHANGE	WHAT CHANGES	VERDICT
✅	Add a field `archivedAt`	signature, extended	**usually safe** (breaks strict clients)
❌	Rename `title` to `name`	signature	**breaks**
❌	Change `id` from number to string	signature (type)	**breaks**
❌	Start returning 25 results per page	contract (behavior)	**breaks**
✅	Add an optional parameter	signature, extended	**usually safe**
➖	Reword an error message	neither (outside the contract)	**often safe** (some clients match the text)

The contract says what *should* be safe; Hyrum's law says someone may still break.



Tools: Interoperability & extensibility

Describe APIs	OpenAPI / Swagger · GraphQL schema
Versioning	semantic versioning · /v1/ paths
Catch breaking changes	contract tests (Pact) · API linting in CI (Spectral)
Extend	webhooks · plugin APIs



One endpoint, six quality attributes

Performance

GET → 50 per page + indexes on repository and open/closed

Consistency

two people edit one issue → reject the stale edit

Reliability

POST → a retry must not open two issues (idempotency key)

GET / POST
/repos/{owner}/{repo}/issues

Gitea: list and create issues

Security

access token scoped to read issues only

Recoverability

new search filter → killswitch + staged rollout

Interoperability

add fields, don’t rename them; webhook on “issue opened”

Every endpoint touches several quality attributes; each move has a cost.

Architectural styles

Bundles of design decisions

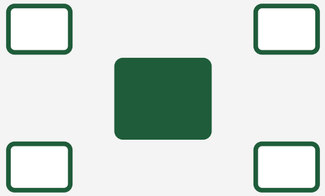
Classic styles at a glance (1/2)



Layered

each layer uses the one below

- ✓ modifiability
- ✓ test each layer alone
- ✗ every change crosses layers
- ✗ some layers just pass data through



Client–server

many clients, one service

- ✓ central control
- ✓ update once for all clients
- ✗ the server is a bottleneck
- ✗ clients need the network



MVC

model / view / controller, organizes the user interface

- ✓ change the UI independently
- ✓ test logic without the UI
- ✗ even small features touch three parts
- ✗ business rules creep into controllers

Styles combine: a typical app is a client–server, layered monolith.

Classic styles at a glance (2/2)



Pipe-and-filter

data flows through steps

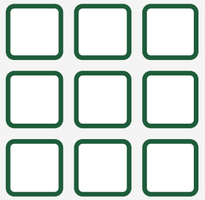
- ✓ reuse, easy to reorder
- ✓ steps run at once on different items (assembly line)
- ✗ poor for interactive work
- ✗ every step must agree on the format



Monolith

one deployable

- ✓ simple to build, test, deploy
- ✓ fast in-process calls
- ✗ any change redeploys everything
- ✗ teams share one codebase and one release

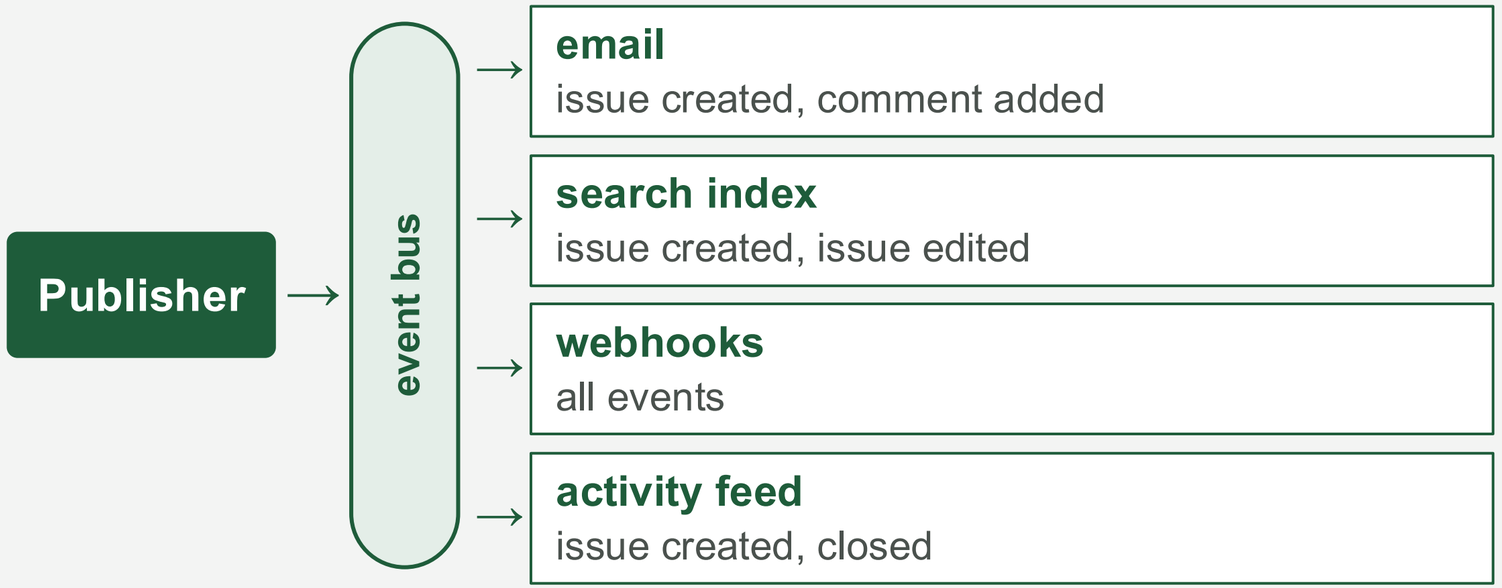


Microservices

many small deployables

- ✓ deploy and scale independently
- ✓ each team owns a service
- ✗ calls can fail
- ✗ many apps to run

Publish–subscribe



Publishers announce **“this happened”**. Subscribers **register** for the event types they handle: one event → many subscribers.

Good for: adding reactions without touching the publisher

Cost: harder to trace; order and delivery need care

At class level this is the **Observer pattern** ; across a system it’s **event-driven / publish–subscribe** .

Publish–subscribe in Gitea

1 Register

At startup, email, webhooks, activity feed and in-app notifications each sign up as **notifiers**.

→

2 Publish

The issue service announces **“issue created”** , once.

→

3 Deliver

Every notifier gets it. Those that handle “issue created” react: send the email, fire the webhooks, add to the feed. The rest ignore it.

Adding a reaction = adding a notifier; the issue code doesn’t change.

Event sourcing



How it works	Store every change as an event; never edit or delete. Current state = replay the events.
Good for	money, audit trails, offline sync (Actual Budget). Git works this way.
Cost	old events live forever, so changing their format is hard; queries need extra read views.
Often enough	a normal table plus a history table (who changed what, when).

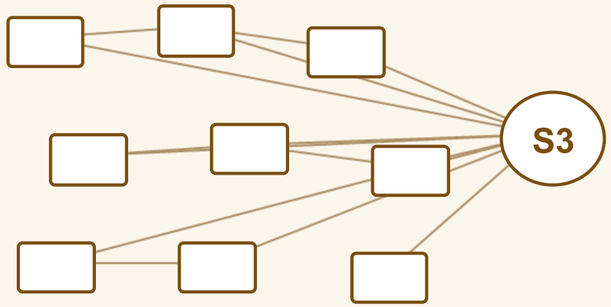
A **data pattern** , not a whole-system style: often used for one part (e.g. the ledger), often combined with publish–subscribe.

Prime Video, 2023

A tool that **watches every live stream** and flags quality problems (frozen frames, audio out of sync).

BEFORE

Lambda: small functions AWS runs on demand · **Step Functions:** coordinates the steps, billed per step · **S3:** file storage between steps



−90%
cost

AFTER

ECS: one containerized app; frames stay in memory

one process
on containers

Every frame = several billed steps + S3 trips → **expensive, hit limits**

Not “microservices are bad”: **the right style at the right stage**

Name the style

A **Gitea:** a request goes from **routers** to **services** to **models** (the database). Each part only calls the one below it.

B **Outline:** when a document changes, the code **announces an event**. Separate **processors** (search, notifications, webhooks) each handle the events they signed up for.

C **Actual Budget:** every change is **saved as a message in a log**. Each device **rebuilds its data by replaying** the messages, even offline.

STYLES

Layered | Client–server | MVC | Pipe-and-filter | Publish–subscribe | Event sourcing

QUALITY ATTRIBUTES

Modifiability | Extensibility | Performance | Offline availability | Reliability | Security

For each: name the style and the quality attribute it mainly serves.

Name the style — answers

PROJECT	STYLE	SERVES
A **Gitea**	Layered	**modifiability**
B **Outline**	Publish–subscribe	**extensibility**
C **Actual Budget**	Event sourcing (close to it)	**offline availability** **sync**

Recording design decisions

The code shows how. The record shows why.

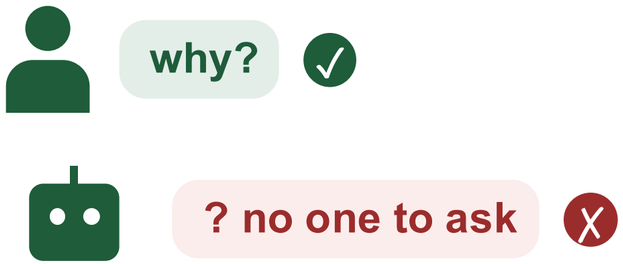
A decision record

Decision	what we chose	
Alternatives	at least two; include *follow the existing pattern* and *the simplest thing*	
Rationale	which quality attribute, and **why**	**law 2 · why**
Consequences	the trade-off and the new failure modes	**law 1 · trade-off**

Why write it down?

“Some context simply is not in the code… I have to ask the developer why it was written this way, but the model has no way to ask.”

— practitioner P1, in Y. Lyu et al., “How Do Practitioners Build SE Agents? Insights from a Mixed-Methods Study,” arXiv:2607.10856, 2026



ALSO

- **People leave:** the reason leaves with them
- **Onboarding:** newcomers learn the why in an hour
- **Know when to revisit:** recorded assumptions show when a decision no longer holds
- **Stop re-arguing:** answer “why not X?” with a link, not a meeting
- **Think it through:** writing down the options forces a real comparison

Improve this decision record

A store’s admin dashboard (e.g. Medusa) shows **sales totals per day** for the last 90 days. It’s the first page owners open, and it takes **4 s** to load. **Requirement:** load in **under 500 ms** (p95); totals up to a few minutes old are fine.

Decision:	Use Redis to cache the dashboard.	?
Alternatives:	none.	?
Rationale:	It’s faster.	?
Consequences:	Faster pages.	?

Rewrite it. What’s missing?

Improve this decision record — a stronger version

Decision:	A job runs every 5 min and saves daily totals to a `daily_sales` table. The dashboard reads that table.	✓
Alternatives:	· Index on (store, date): 4 s → 2.5 s, not enough · Redis cache: a new service; slow loads after each expiry	✓
Rationale:	Dashboard `p95 is 4 s; target < 500 ms` . Totals a few minutes old are fine.	✓
Consequences:	Totals `up to 5 min old` ; one more scheduled job to monitor	✓

Why the job wins

- **No user waits:** the slow query runs in the background
- **No new service:** uses an existing table
- **Nothing to clear:** the job overwrites the totals

Agents and design

Fast code, slow erosion

What to watch for in agent-written code

! Handling instead of removing

```
try {
  try {
    ...
  } catch {}
} catch {}
```

try/catch, retries, fallbacks

workflow step 2

more code, same problem

! New helpers instead of reuse

fmtDate()

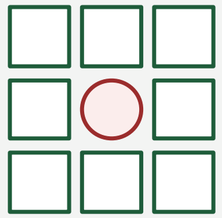
formatDate()

duplication

DRY

two of everything

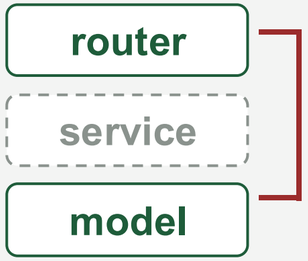
! Locally clean, globally inconsistent



consistency

fine alone, off-style together

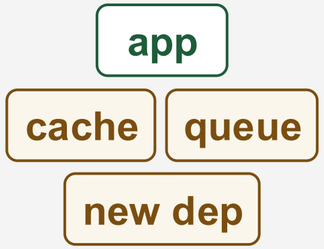
! Layer shortcuts



layered

skips the rules

! Extra moving parts



YAGNI

more to run

Erosion in one plan

Requirement: a booking can’t start less than 2 hours from now.
The agent’s plan:

1. Put the 2-hour rule in `BookingRepository`
   ✗ **business rules belong in services**
2. Add a new key helper
   ✗ **a third copy (two exist)**
3. try/catch, retry 3×, ignore errors
   ✗ **no idempotency key; errors hidden**
4. Add a Redis cache for slots
   ? **needs a reason**

CAL.DIY · AGENTS.md

> Never put business logic in repositories - that belongs in Services



Guardrails for design

1 **Write the rules down**
AGENTS.md / architecture docs

Cal.diy: “Never put business logic in repositories”

Actual Budget: “Don’t directly import another platform’s modules”

Outline: “Refer to /docs/ARCHITECTURE.md”

2 **Check them automatically**
lint rules, dependency checks

Gitea: a lint rule (depguard) **fails the build** if code imports Go’s standard JSON library instead of Gitea’s own wrapper

3 **Review the design, not just the diff**



Ask: Fits the existing style?

Removed or handled the problem?

Every new part justified?

Who owns the design?

Agent

explores, proposes options, drafts the record

options

You

choose, explain why, own the consequences

decision → **consequences**

The decision record is where this ownership is written down.

An agent has no skin in the game. You do.

Takeaways

1. Architecture = structure + decisions + **why**. Quality attributes drive it.
2. Low coupling, high cohesion, hidden decisions.
3. Everything is a trade-off. Prefer the simple design.
4. Remove the problem before you handle it.
5. Styles are bundles of decisions: name what they serve.
6. In an existing codebase, follow its style.
7. Write decisions down. Agents propose; you own them.